

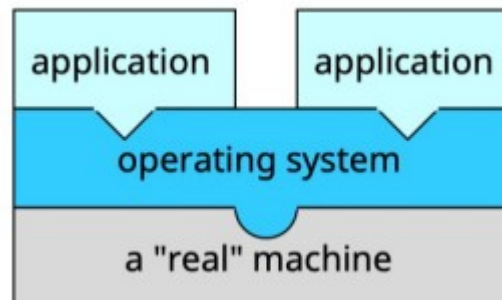


Cloud computing virtualization and containerization



System and virtual machine virtualization

Up to 20 years ago, the software structure was developed in the following way:
the *“real” machine*, the *operating system* and the *applications* on top of the OS.



For the quality of the software, the real *issue* of this development was the *portability*: it is not guaranteed that using a product on different architectures (*hardware + operating system*), it can run.



System and virtual machine virtualization

The software depends on the operating system and hardware. Why?

- *The software is compiled by a low-level programming language that is typically dependent on the specific **hardware** platform.*
- *The software should use some services of **operating system** (e.g., system call, Windows manager, ...).*



Virtual machine and virtualization

To solve the issue of portability, it comes the concept of virtualization.

*The **virtualization**, in general, is intended to provide access to a set of virtual **computational resources*** starting from a set of real computational resources. For example:*

- *Virtual Private Network*
- *File System*

***Resource virtualization** takes place based on a **virtualization software layer**.*

**IO, CPU, memory, physical disk*



System virtualization

The *virtualization system* uses software to create an abstraction layer on the hardware of an entire physical system. This allows to divide the hardware elements of a single computer (processors, memory and storage) into several virtual computers, the *virtual machines*.

It is an important technology, especially in *distributed systems* and in the *cloud*:

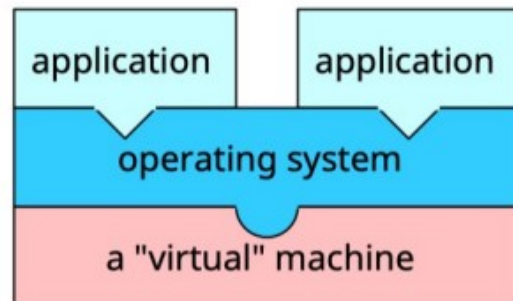
- *It is an enabling technology for the flexible management of (virtual) execution environments.*
- *It is an enabling technology of cloud computing.*
- *It can sustain qualities such as availability and scalability.*



Virtual Machine

The **virtual machine** is a software layer that gives the impression, to the operating system and applications above it, of being a “real” machine: it is an **emulation**.

Virtual machines have the same interface as a real computer, which is a set of (virtual) hardware resources such as processors, memory, storage, networks devices.

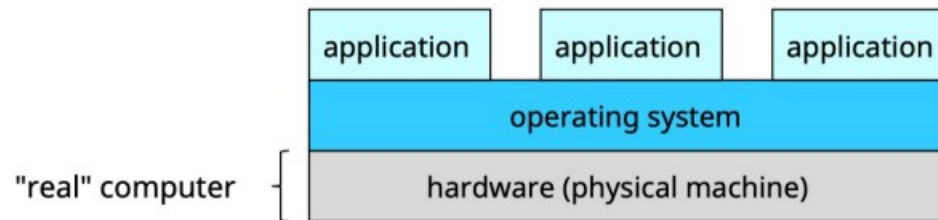


A virtual machine provides the virtual hardware of a complete computer, where you can install and run an OS and a set of services and applications.



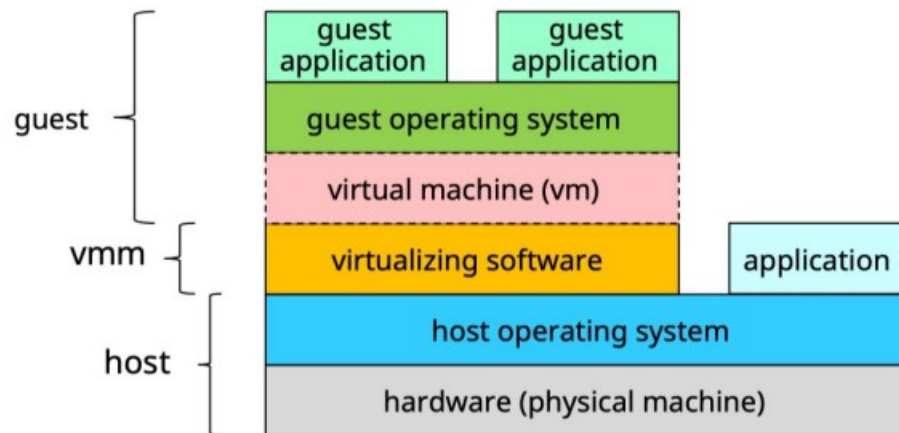
Virtual Machine

Non-virtualized computer



There is an hardware, the operating system and the different applications that run on top of it.

Virtualized computer



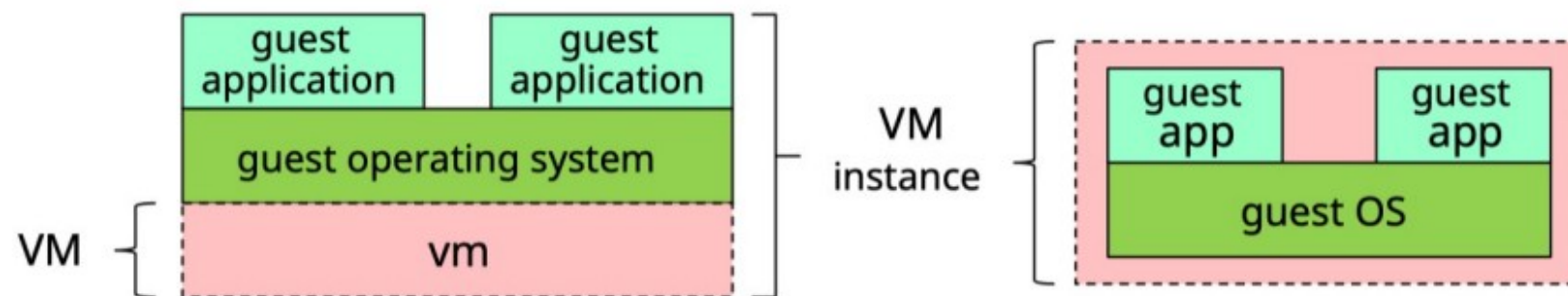
*There is an hardware, the host operating system, virtualizing software (**hypervisor** or **virtual machine monitor**), one or multiple virtual machines (instances of this virtual computer), guest operating system and the guest application.*



VMs and VM instances

A *virtual machine (VM)* is a virtual entity that emulates the *hardware* of a real computer.

A *virtual machine instance (VM instance)* is a virtual machine along with its OS, applications and state.





Hypervisor

Virtual machines are coordinated by the **hypervisor**, which is the software layer. It serves as an **interface** between the VM and the underlying physical hardware, ensuring that each has access to the physical resources it needs to execute and that VMs don't interfere with each other.

There are two main types of hypervisors:

- **Type 1 (native VMM or bare-metal hypervisor)**
e.g. VMware vSphere, Xen, ...
- **Type 2 (hosted VMM or hosted hypervisor)**
e.g. VMware Workstation, Oracle VM Virtualbox, ...



Hypervisor

The hypervisor must provide the *illusion* that each VM acts like a real computer.

The effectiveness and efficiency of these illusions is possible with the following characteristics:

Fidelity

The behavior of a program in a VM should match the behavior as if it would run in a physical machine.

Safety

The hypervisor should have complete control of the virtualized resources.

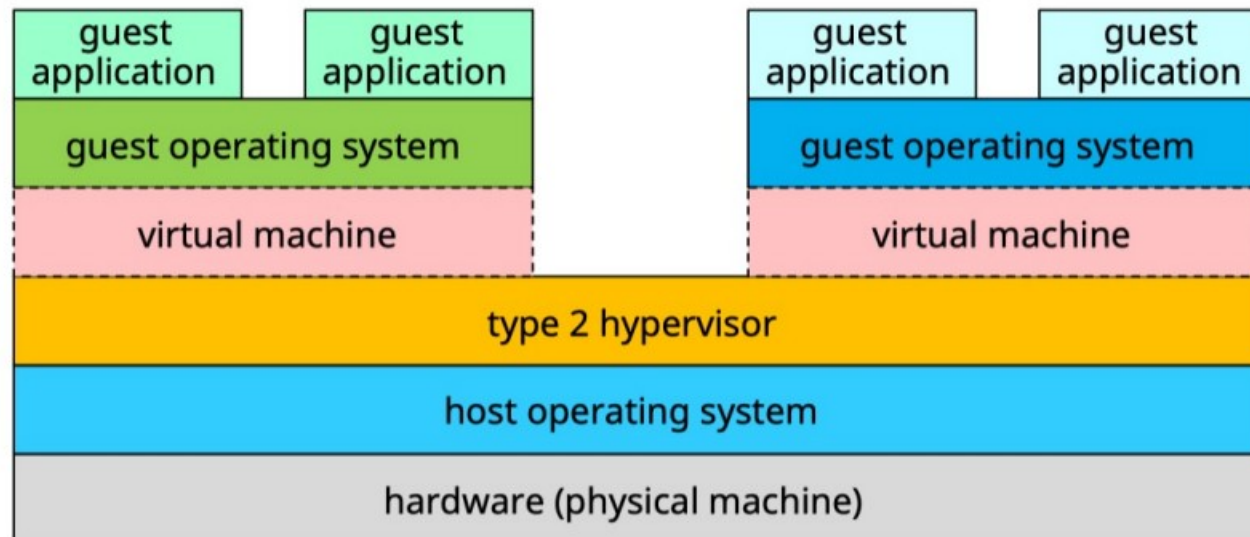
Efficiency

Most of the VM code should be executed directly from the host computer, without the intervention of the hypervisor.



Hypervisor type 2

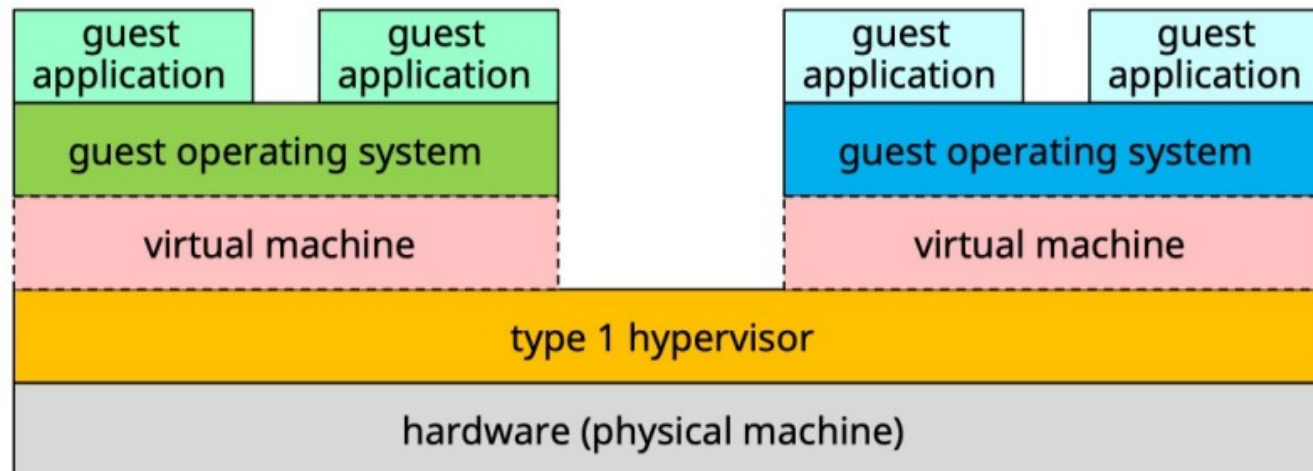
From the point of view of the host, it is an application that should be installed and run on top of an existing host operating system.





Hypervisor type 1

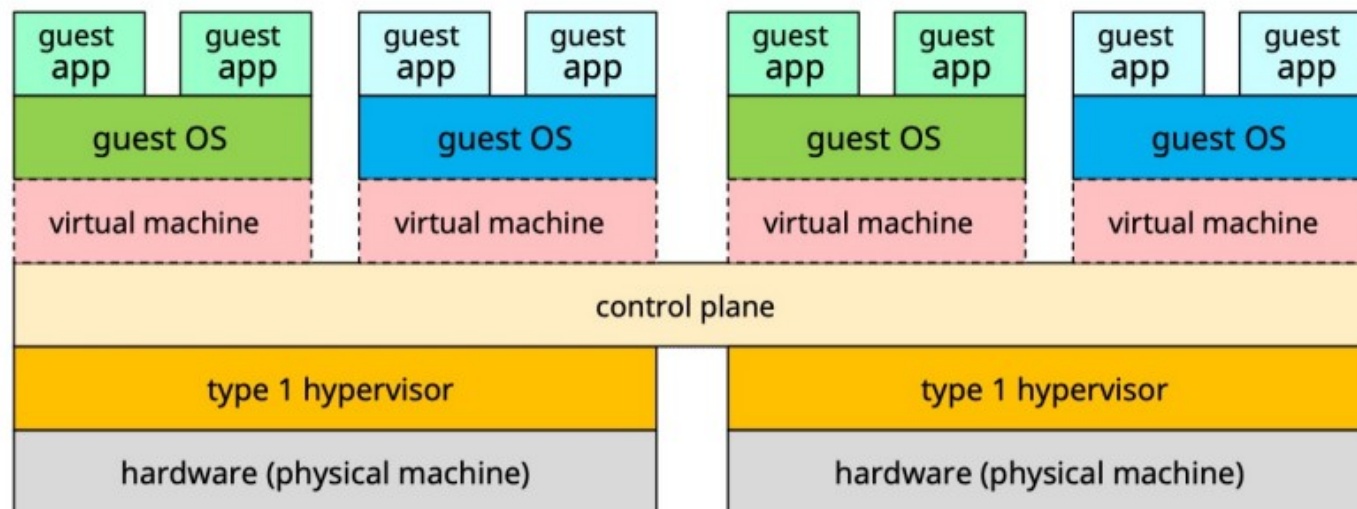
It interacts with the underlying physical resources, replacing the traditional operating system entirely.





Hypervisor type 1

*The use of a distributed software layer (**control plane**), which is a small layer on top of the hypervisor, allows type 1 hypervisors to virtualize a cluster of physical hosts, giving the impression of a huge machine.*

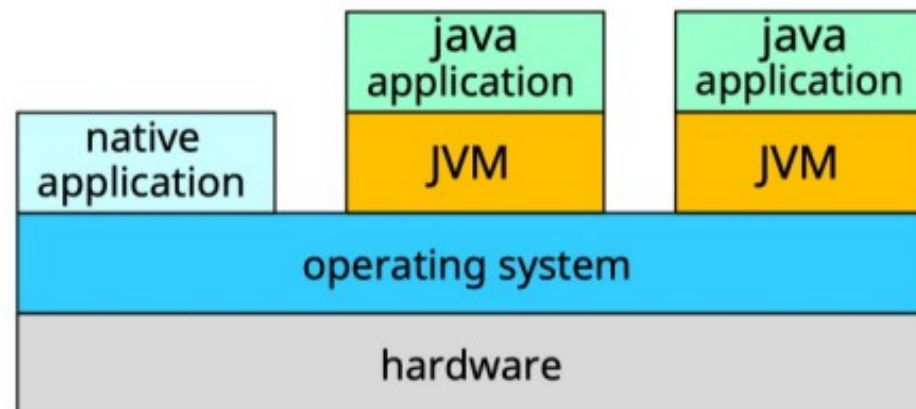




Process virtualization

***Process virtualization** is another form of virtualization.
e.g. The Java Virtual Machine (JVM)*

On top of the operating system, you have a system that gives you the impression that one process of the operating system can be seen as a computing resource.



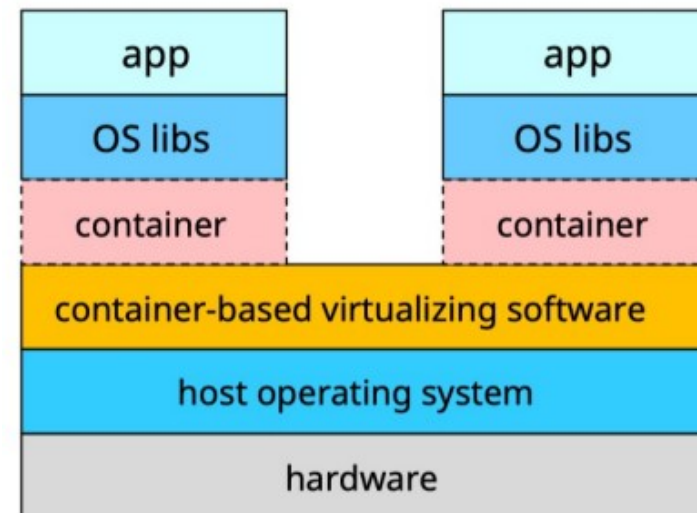


Container-based virtualization

The virtualization based on container (*container-based virtualization* or *OS-level virtualization*) is another form of virtualization.

This type of virtualization is *more powerful* of the process virtualization.

The containers share an underlying OS kernel.





Techniques of System virtualization

The hardware of a computer (real or virtual) consists of a set of resources (real or virtual) - CPU, memory, I/O, networks, storage, ...

*For each type of resource there are different **virtualization techniques** used in the system virtualization:*

- *Processor virtualization*
- *Memory virtualization*
- *I/O virtualization*
- *Storage virtualization*
- *Network virtualization*



Processor virtualization

A **processor** is characterized by the **ISA** (*Instruction Set Architecture*) that defines its instructions and its state (registers and memory).

There are two approaches that can virtualize a processor:

- **Processor emulation**: takes place through ISA virtualization (CPU emulation) and is based on **binary translation** of instructions. Due to the widespread use of X86 processors, this technique was no longer important for several years, it is getting back nowadays with ARM processors.



Processor virtualization

- *Processor virtualization*: if the real processor and the virtual processor are of the same type, then many software instructions running in a VM can be executed directly from the **host processor**.

Some instructions cannot be performed directly from the host processor. For example, some of the guest OS kernel instructions:

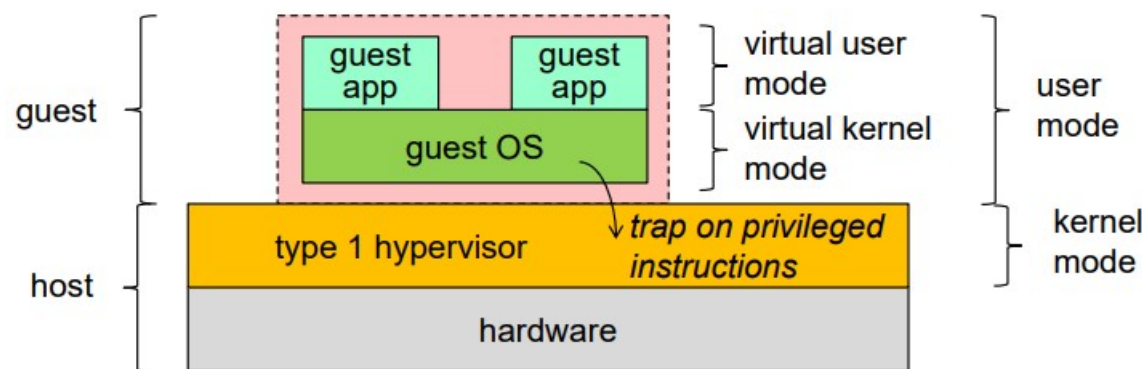
*E.g. instructions to enable and disable interruptions and instructions for managing the **Memory Management Unit (MMU)**.*



Hardware-assisted virtualization

The *hardware-assisted virtualization* (or *trap-and-emulate*) is the main processor virtualization technique.

The VM instructions are executed by the real processor of the host, which normally runs them directly, but which captures non-virtualizable VM instructions ("*trap*") and manages them as hypervisor calls ("*emulate*").



It is based on the use of virtualization hardware technologies (ex. *Intel VT-x* and *AMD SVM* in x86 processors).



Full virtualization and Paravirtualization

Additional processor virtualization techniques:

- **Full virtualization:** it operates by combining *binary translation* (of “problematic” instructions) and *direct compilation* (of most of the instructions), with the guest OS being completely isolated from the virtualization layer and hardware by the virtual machine.
e.g. Microsoft and Parallels systems
- **Paravirtualization:** It uses *hypercalls* for operations to handle instructions at compile time, with the guest OS partially isolated from the virtualization layer and hardware by the virtual machine.
e.g., VMware and Xen



Virtualization of multiple processors

*The physical processors (with their cores) of the host are virtualized in virtual CPU (**virtual CPU** or **vCPU**) and assigned to the VMs.*

- *Each vCPU has only one core*
- *Each VM can be assigned one or more vCPU*

*In this way, a **multi-processor system** is virtualized in a **multi-computer system**.*

*When applied to multi-core processors or multi processor computers, the processor virtualization is particularly **effective**.*



Memory virtualization

*In system virtualization, you need to manage a double level of physical **memory virtualization**, because each VM has its own virtual memory.*

*Starting from 2008, processors from **Intel** and **AMD** began to provide hardware-level software to manage the switching between virtual memory and physical memory.*

*Efficient memory management is also achieved thanks to some techniques made available to hypervisors:
e.g. page deduplication and ballooning*



I/O virtualization

As we have seen, the hypervisor is responsible for managing and allocating hardware resources to VMs.

***I/O virtualization** enables secure and flexible interaction of virtual machines (VMs) with **input/output** (I/O) **devices**.*

*The hypervisor, instead of directly assigning I/O hardware devices to VMs (disks, network cards, ...), physically present on the host, assigns **virtual devices** to each VM.*

These virtual devices are separately configurable for each VM and may be different from those physically present on the host.



I/O virtualization

*The hypervisor can also assign VMs to virtual devices that do not have an equivalent physical counterpart (e.g. **virtual switch**), to which the guest OS accesses through its own drivers.*

Within the VM, the guest operating system accesses these virtual devices using its own drivers. This means that the guest OS treats these devices as physical, not virtual.

*The **input/output operations** for these virtual devices are captured by the hypervisor, which handles them appropriately.*



Storage virtualization

Storage virtualization aims to abstract the physical layer of storage from that of VMs.

Storage virtualization can be applied:

- *To units directly connected to the host computer*
 - ***DAS** (Direct Attached Storage).*
- *To units connected to a network*
 - ***SAN** (Storage Area Network):* provide block access in remote units.
 - ***NAS** (Network Attached Storage):* provide access to files in remote drives.

SAN/NAS-based virtualization increases flexibility.



Storage virtualization

Here are some possible *storage virtualization* options:

- A virtual disk that corresponds directly to a physical disk or its partition.
 - *Best performance.*
- A virtual disk implemented as a set of host files (*disk image files*).
 - *More flexibility.*

When there is the *hypervisor of type 2*, one functionality is the *shared folders*: you have a virtual folder on the virtual hard disk of the guest OS corresponding to the physical folder of the physical hard disk of the host OS.



Network virtualization

Physical network:

- A set of physical hosts, each with one or more *physical network cards* (**pNIC**).
- One or more *physical switches* (**pSwitch**), to connect multiple physical network segments.

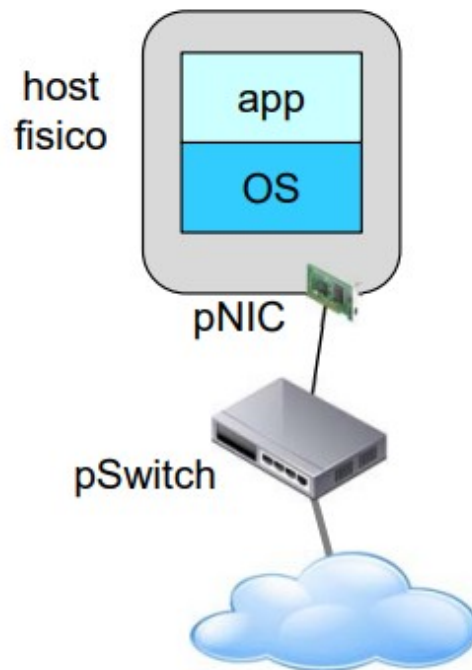
Virtual network:

- A set of VMs, each with one or more *virtual network cards* (**vNIC**).
- One or more *virtual switches* (**vSwitch**), to connect virtual network cards (**vNIC**) to each other and with physical network cards (**pNIC**).

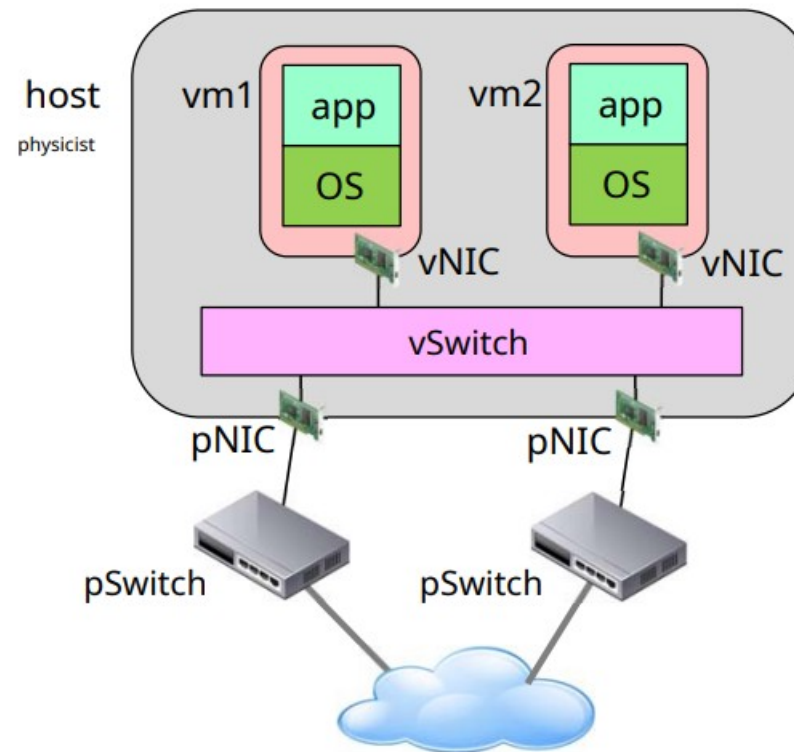


Network virtualization

Physical Network



Virtual Network (type 1 hypervisor)





Network virtualization

As we have already seen, *virtual machines* consist of one or more *virtual switches* and one or more *virtual network cards (vNIC)*.

Each *vNIC* can emulate a common *real network card*.

Paravirtualized vNICs (e.g., virtio-net) are often used to improve performance.

The IP address of the virtual network cards can be configured statically or dynamically via *Dynamic Host Configuration Protocol (DHCP)*.



Network virtualization

Each vNIC can operate differently than other virtual network cards. Below are the different modes:

- **NAT (Network Address Translation):** *the guest VM sees the external network via the vNIC.*
- **Bridged Networking:** *vNIC is connected to a pNIC, and exchanges packets with external network (e.g., Internet) directly through it.*
- **Internal Networking:** *vNICs allow to connect a group of guest VMs to each other and create a network of VMs.*
- **Host-only Networking:** *vNIC allow to define a network that contains the host and a set of guest VMs.*



Network virtualization

The **port forwarding** matches a port of a guest VM with a port of the host.

It's a technique used to make services, running in a guest VM, accessible to the host or external network.

For example, the **guest's port 80 (HTTP)** is connected to the **host's port 8080**. This way, when something will reach for port 8080 of the host, it will be redirected to port 80 of the guest.



Virtual machine instances

A *virtual machine instance (VM instance)* is a dynamic entity with its own state, which can change over time.

An instance of VM can actually be running on a certain host.

When we talk about the state of an instance of VM, we mean the state of all its resources in a certain moment of time, that is the state of its disks, memory and registers of its vCPU.



Virtual machine images

*A **virtual machine image** (**VM image**) is a static entity.*

An image of VM can't be running but can be used to run an instance.

*A VM image is formed by the **VM metadata** (e.g., number of vCPU, amount of memory and MAC address of network cards) and the content of the **VM volumes/disks**.*

Moreover, a VM image can be represented by an appropriate format of one or more files.



Virtual machine images and instances

*What is the relationship between **VM images** and **VM instances**?*

- *An instance of VM can be created from a VM image.
E.g., on Amazon EC2, you can create a VM by selecting an instance type (ex, **A1.large**) and a VM image, the Amazon Machine Images (AMI) (e.g., the **AMI Linux Ubuntu 18.04 for x64**).*
- *The state of a VM instance can be saved as a VM image.
Ex., to easily create new VMs from that image.*



VM cloning

The **VM cloning** consists in creating a new or more VM instances from a VM image.

This is **not** a simple **VM image copy**. In fact, cloning a virtual machine means creating a complete duplicate of the VM configuration, including hardware settings, network configurations, virtual machine information, ...

This allows us to have two (or more) identical virtual machines, each functioning as a separate VM, with their **own system configurations and networks**.



Virtual appliance

A **virtual appliance** is an image of **pre-configured VMs** (usually from third parties) from which you can create instances of VMs with that configuration (e.g., you already have the version of operating system, ...).

So, it's a VM already configured for a certain type of usage. Therefore, the availability of virtual appliance can significantly **reduce** the creation time of VMs.

Pre-configured VM images are typically made accessible in a public or private repository, in an appropriate format (e.g., VMware VMX or VirtualBox VDI).



Snapshot/checkpoint

*A VM can be **started**, **stopped**, **paused** or **restarted**.*

*When a virtual machine is **paused** or **stopped**, its state can be saved as a **snapshot** (or **checkpoint**) for future use. When we talk about state we mean the state of the disk, the state of the memory and the state of the registers of the processors.*

It is also possible to start a VM from a snapshot, so as to reduce the startup time of a VM.



Migration of VM

The **migration of VM** aims to move a running VM instance from one physical host to another. Two ways to migrate:

- Turn off the VM in the first host and restart it in the second host.
 - *Not always acceptable*
- Pause the VM, take a snapshot, copy it to the second host and restart the VM on the second host from the snapshot.

- *Better way*

If the snapshot is saved to a **SAN/NAS** drive shared by hosts, copying can be avoided. Also, if the VM storage is managed on a shared SAN/NAS drive between hosts, the snapshot can be limited to the memory state only. In this way, migration can be extremely fast (**live migration**).



Interfaces for VM management

VM management operations, such as creation, configuration, startup and shutdown, can be managed in different ways:

- Manually, via a **GUI** (e.g., VirtualBox) or **web console** (when interacting with a remote data center).
- Via a **CLI** or **REST interface**, including scripts: you have the impression to have a local prompt in the machine. In the reality it is a program that via REST interface allow to send the commands to the remote data center.
 - This mechanism supports **automated VM management**.



Consequences of System Virtualization

- *It is possible to faithfully run an application or service in a VM.*
- *There is an overhead on performance which, however, can be kept low.*
- *Virtualization can support some qualities:*
 - ***Flexibility***: *enables flexible release of distributed software systems into virtual environments.*
 - ***Security***: *virtual machines are isolated from each other and from the host.*
 - ***Availability***: *Virtualization enables quick creation and startup of VMs.*

Here are some virtualization systems for the x86 platform:



Xen

Xen is an open source type 1 Hypervisor for x86 systems.

Born as a research project in the late 90's. In 2002 it became an open-source project.

Starting in 2013 it becomes a "collaborative project" of the Linux Foundation, whose members include Amazon, Google, Oracle, Intel and AMD.

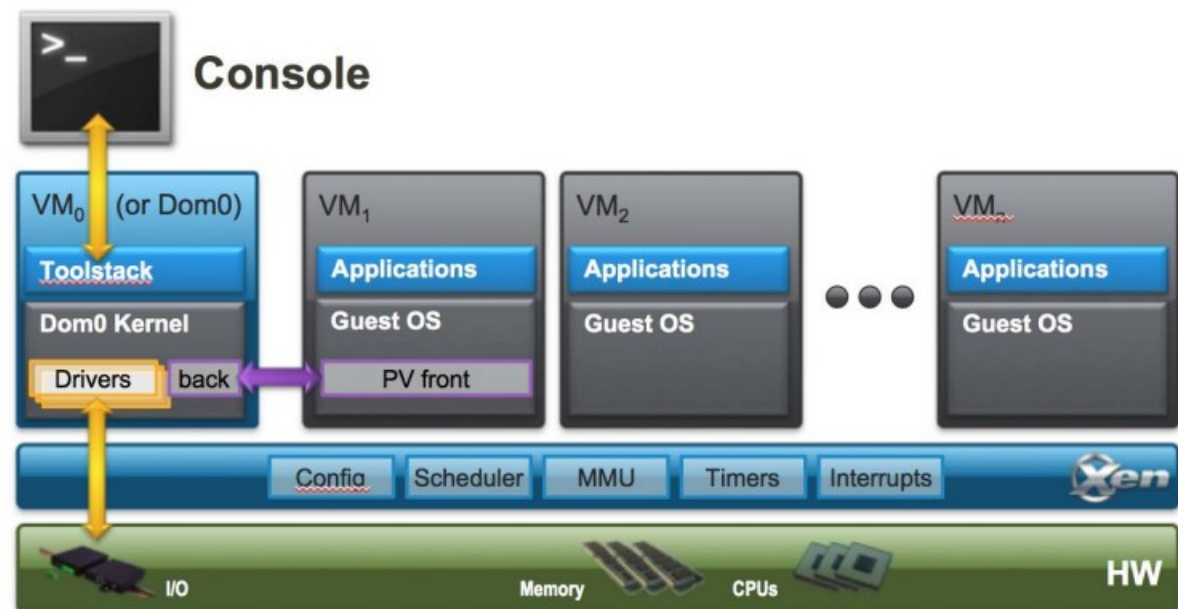
Xen is known for its support of multiple guest operating systems, including Linux, other Unix and Windows systems.

*It supports both **paravirtualization (PV)** and **hardware-assisted virtualization (HVM)**.*



Xen

Above the hardware layer (I/O, Memory, CPUs), there exists the **Xen layer**, a software framework responsible for hosting the operating system. On top of Xen, there are the virtual machines, called guest or domain, each housing a guest OS and applications.





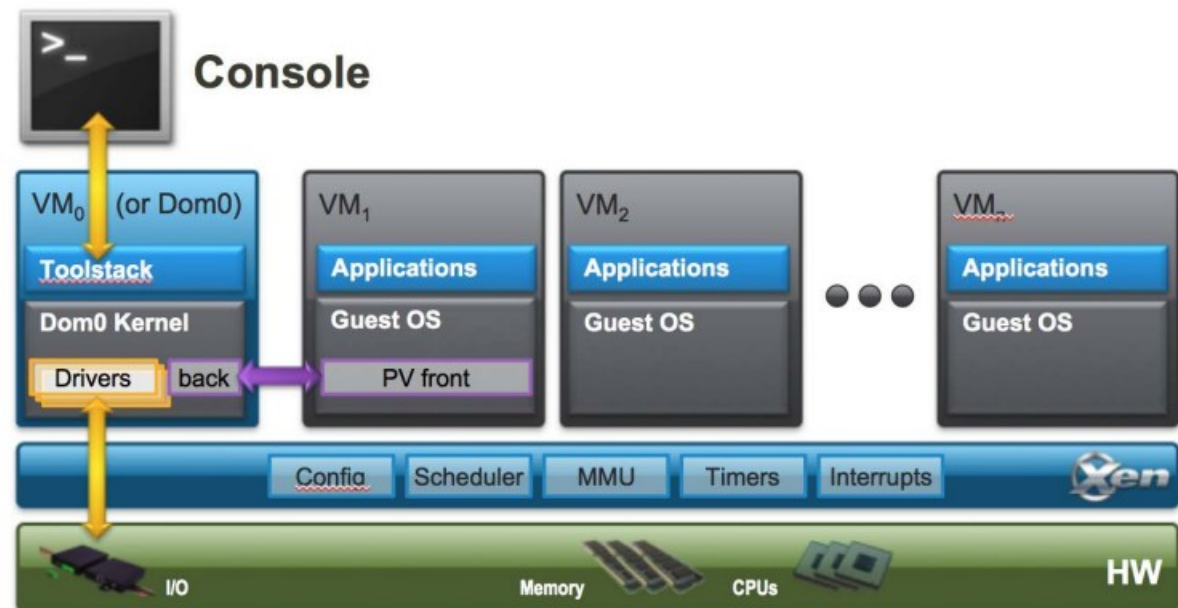
Xen

VM0 is a particular VM: its kernel is the only that works physically via drivers with the I/O.

The **Toolstack** is a control software stack to manage the creation, configuration and destruction of other VMs.

The **back module** allows interactions with different virtual machines.

Within the VMs, the **Pv front module** allows interaction specifically with VM0.



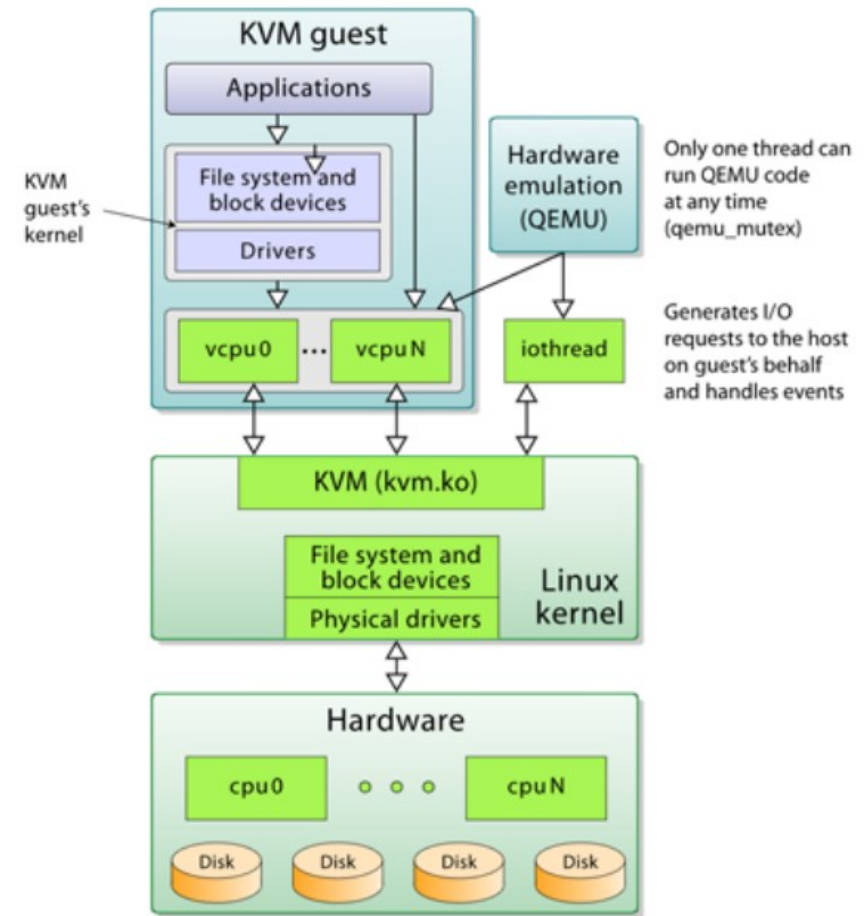
KVM



SAPIENZA
UNIVERSITÀ DI ROMA

***KVM (Kernel Virtual Machine)** is an open-source virtualization solution for x86 systems integrated into Linux kernels. It can be considered a type 1 hypervisor. It supports unmodified Unix or Windows guest operating system.*

*KVM architecture is based on a Linux kernel module (**kvm.ko**) that provides the core of the virtualization.*





*One of the key features of **KVM** is its integration with **QEMU**, a hosted hypervisor that enables hardware virtualization, which relies on binary translation to create an execution environment for guest virtual machines.*

Each virtual CPU (vcpu) of the VMs is managed as a thread within the host operating system, allowing high efficiency and optimized performance.

*Also, it is possible to interact with the virtualization capabilities of KVM through **libvirt**, a common **Linux API** designed to manage and control virtual machines securely and remotely.*



VMware

VMware is a company (subsidiary of **EMC**) with a rich offer of virtualization technologies for small, medium and large companies, which includes:

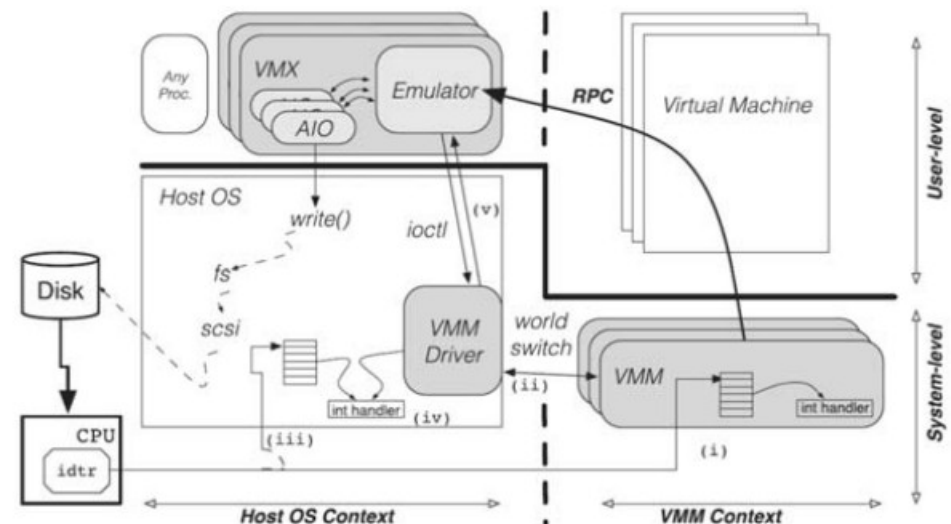
- Virtualization products for individual computers, such as **VMware Workstation** and **Fusion type 2 hypervisors** (Pro and Player).
- Products for data center virtualization and cloud management, such as **vSphere** and **vCloud suite** (includes features for VM availability, automation and management, to provide a private cloud).
- Desktop virtualization products, such as **Horizon**.



VMware Workstation

The **VMware Workstation** architecture is based on three main components (the figure shows the host OS context on the left and the hypervisor context on the right).

- **VMM** (*virtual machine monitor*) is the hosted hypervisor.
- **VMX** is the user interface in the host system.
- The **VMM driver** is installed as a driver in the host OS, but actually drives the VMM and hides it from the host OS.





VirtualBox

Oracle VM VirtualBox is a virtualization product for x86 systems for enterprise or personal use (since 2007).

It is a type 2 hypervisor, for OS hosts Windows, Linux and MacOS, and for guest OS Windows and Linux.

It is an open-source project controlled by Oracle and supports numerous virtualization techniques and options.

In addition, Oracle VM VirtualBox provides access to VMs, which can be created via a GUI or via a command line interface (VBoxManage), both locally and remotely. One of the common applications of this versatility is the use of pre-built virtual machines for developers, thus simplifying the development environment.



Applications of virtualization

*About 20 years ago, a distributed software system consisting of multiple services and servers was in use, each of which was running on a different (physical) computer. It was a good idea but **expensive** and **difficult to manage**.*

*With the birth of virtualization, it began to develop the **server consolidation**: several servers run in different VMs, in one or more virtualized physical computers.*

*This solution turns out to have some positive aspects: the hypervisor guarantees the **isolation** between the different VMs, leading to a greater use of resources and greater **flexibility**, but also to significant **savings**. However, hardware **reliability is deteriorating**.*



Applications of virtualization

Other virtualization applications:

- ***Application consolidation**, which consists of providing an execution environment to a legacy application after migrating to a new hardware/software platform.*
- ***Creation of multiple execution environments**, each with its own OS and its own software stack*
 - *to support the development of distributed software systems*
 - *to support testing and **QA** (**quality assurance**), in multiple and separate environments*



Applications of virtualization

- *Sandboxing*: run unsafe applications.
- *Desktop* (client) *virtualization*:
 - *allows users to access their virtual desktop from any computer (client).*
- *In the hosting of web services.*
- *In the context of cloud computing.*



Benefits of virtualization

- *Cost reduction.*
- *Improvement of application quality:*
 - *availability and tolerance to failures*
 - *IT efficiency, agility, productivity and flexibility*
 - *insulation and safety*
 - *extend the life of applications*
- *Simplification of data center management:*
 - *simplified and faster resource and VM provisioning*
 - *support for scalability and elasticity*
 - *centralized management*
 - *datacenter defined by software*
- *Development, testing and QA support.*
- *Reduce vendor lock-in and drive cloud migration.*



Virtual machines and software release

*VMs are a release option for distributed software systems. Each VM encapsulates one or more **software services**, along with the software stack needed for those services.*

How to manage these VMs?

- ***“Traditional” approach:** each VM is created manually, installing the software and services of interest manually. VMs are essentially considered to be the virtual version of a physical computer.*
- ***A modern and better approach:** The images of the VMs of interest are built automatically. Next, the VMs are created from these images.*



Benefits

Benefits of using VMs for software release:

- *The release of a service, which is managed as the creation of a VM from the image related to the service itself, is simple and reliable.*
- *Each VM (with its services) is run in isolation.*
- *VMs can be released both on the cloud and on premises, in a private data center.*
- *Creating and starting a VM takes little time.*



Drawbacks

***Drawbacks** to using VMs for software release:*

- *Resource **inefficiency** is possible. In fact, every service or group of services requires an entire VM. This inefficiency increases if each VM is used for a single lightweight service but the release of multiple services in one VM reduces fault isolation.*
- ***Overhead** in the system administration of VMs: who creates the VM (or its image) is responsible for also carrying out updates of the software that is installed there*
- *Creating and starting a VM (starting with a VM image) typically takes a few seconds to minutes.*



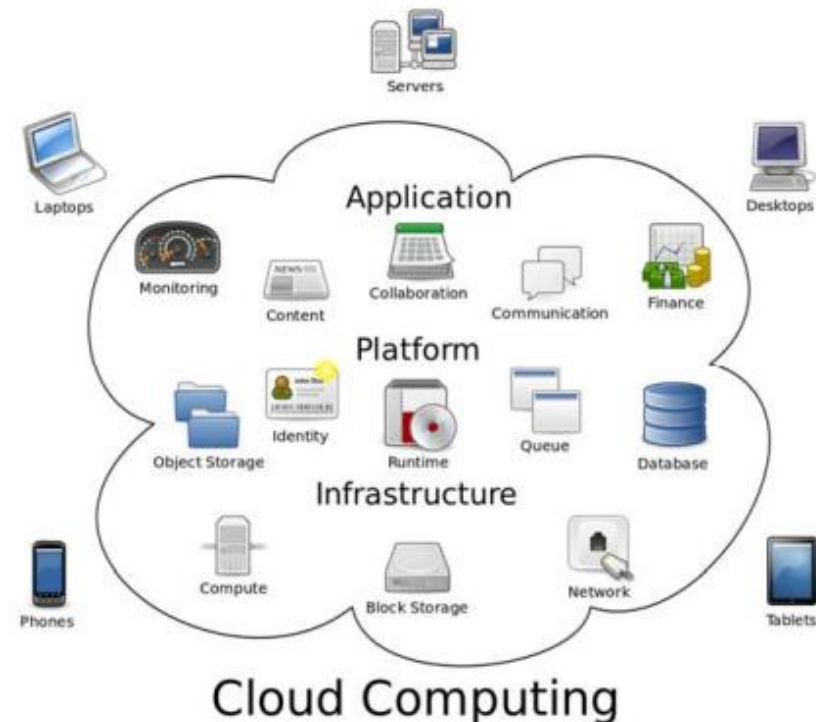
Cloud computing virtualization and containerization



Introduction to cloud computing

Cloud computing comes from an old idea: computing (computational resources) as a utility service.

In the center there is an infrastructure that is the utility provider. All the devices around can exploit the computer resources of the infrastructure in a plug-and-play way.





A definition of cloud computing

Cloud computing is, according to the **NIST** definition, a computing model that enables network access, on demand, **ubiquitous** (anywhere, anytime) and convenient to a pool of computing resources (*CPU, storage, networks, operating systems, services and/or applications*) shared and configurable that can be acquired and released quickly and dynamically with minimal management effort, or in any case with minimal interaction with the service provider.

In addition, this processing model provides five essential features, three **service models** and four **deployment models**.



Service models

The main **service models** in cloud computing are:

- **Software as a Service (SaaS)**

It is a software distribution model where applications are hosted by a service provider and made available to users through the internet.

Some examples:

- Google Workspace (Gmail, Google Docs, ...)
- Microsoft Office 365
- Netflix
- Salesforce.com applications (e.g., CRM)



Service models

▪ Platform as a Service (PaaS)

PaaS services provide a comprehensive development and execution environment for applications.

Some examples:

- Google App Engine
- Amazon Elastic Beanstalk
- Service App Microsoft Azure

▪ Infrastructure as a Service (IaaS)

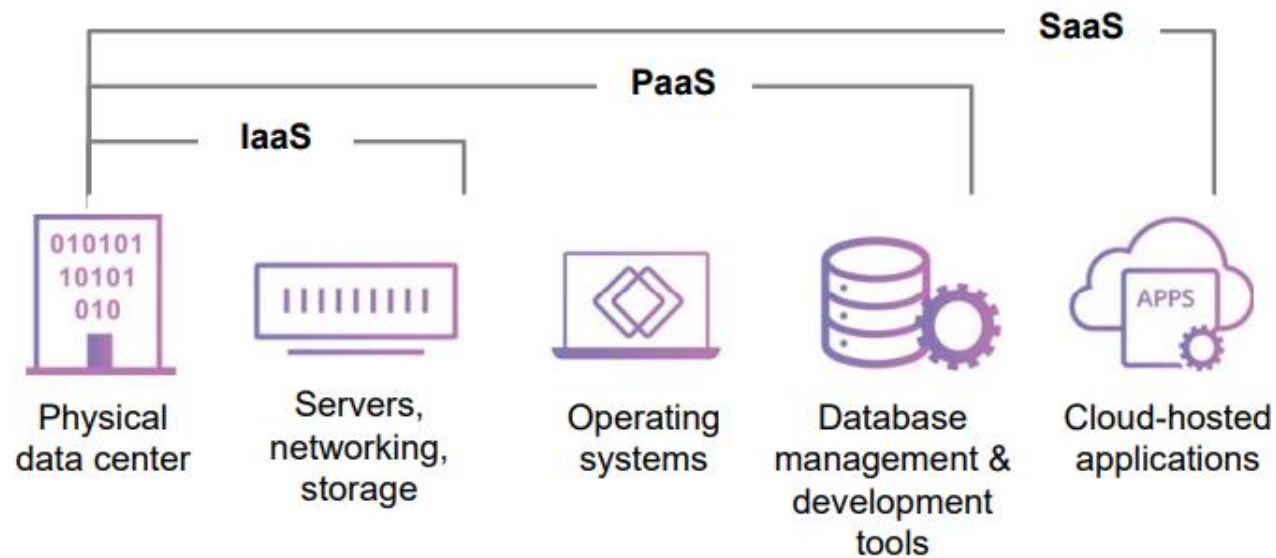
IaaS services provide virtual access to hardware resources, such as servers, networks, storage, and other infrastructure components.

Some examples:

- Many Amazon Web Services (AWS) products

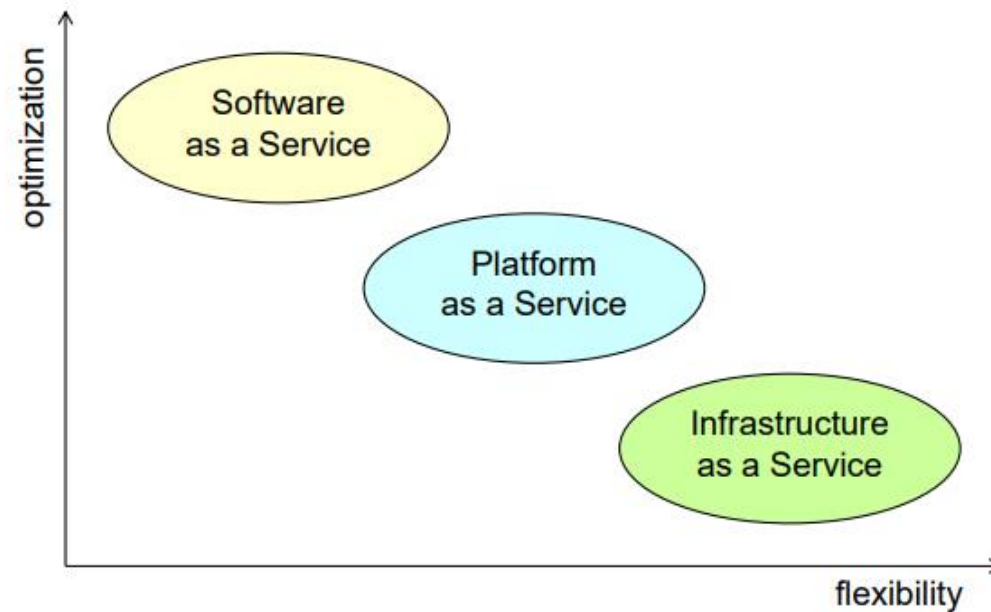


The SPI model





The SPI model





Deployment models

Cloud computing includes four **deployment models**:

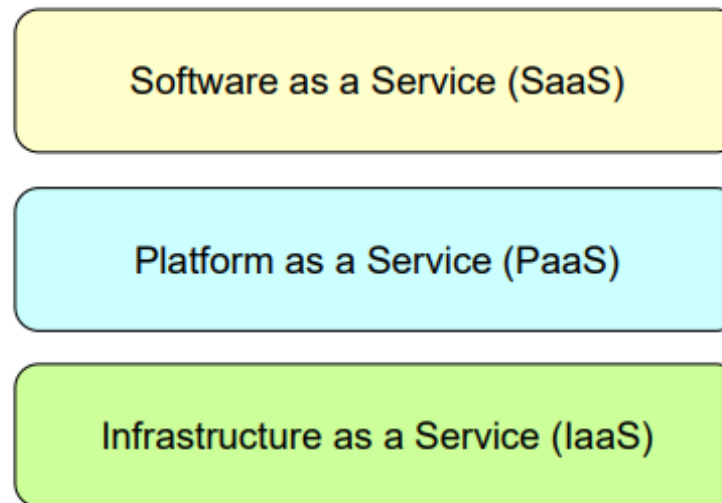
- **public cloud**: the cloud infrastructure is made available to the public
- **private cloud**: cloud infrastructure is managed for a single organization
- **hybrid cloud**: cloud infrastructure is the composition of two or more clouds
- **community cloud**: cloud infrastructure is shared by multiple organizations





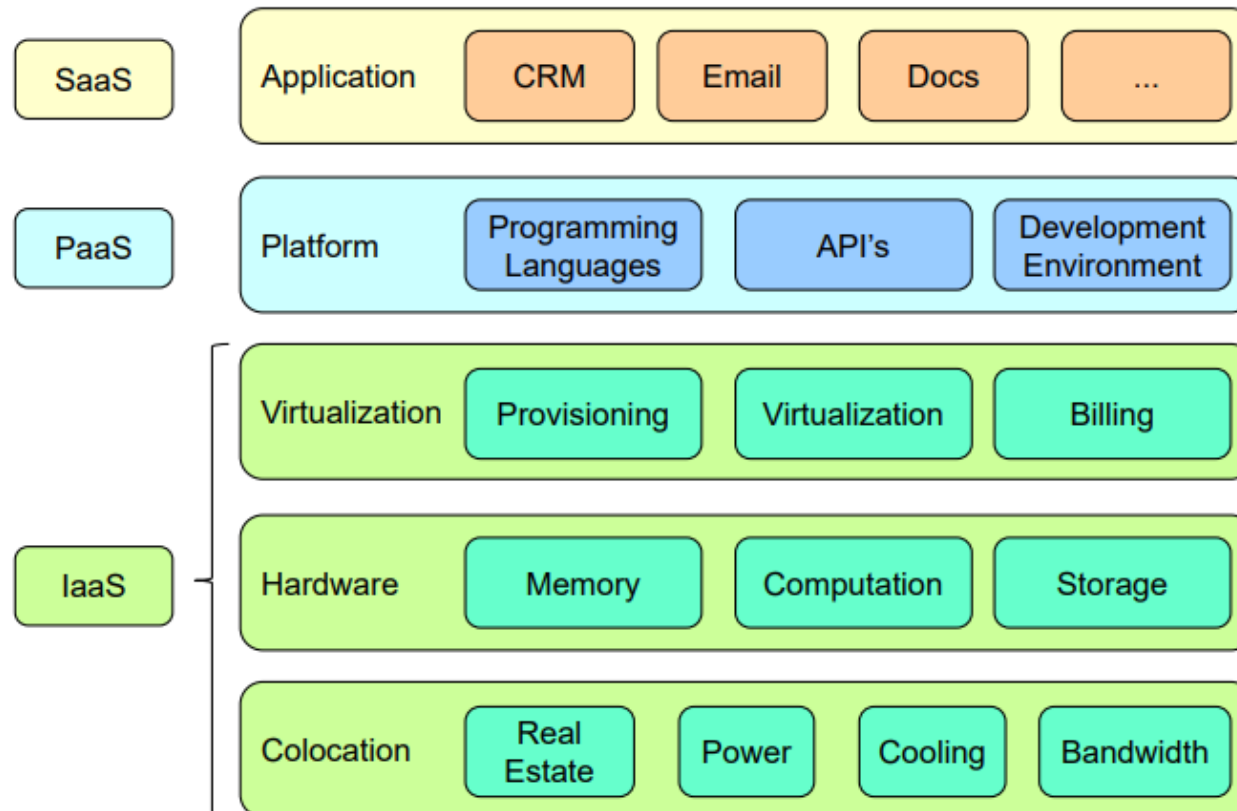
Cloud architecture

You can think of the cloud in terms of a layered architecture, referring to the SPI model:





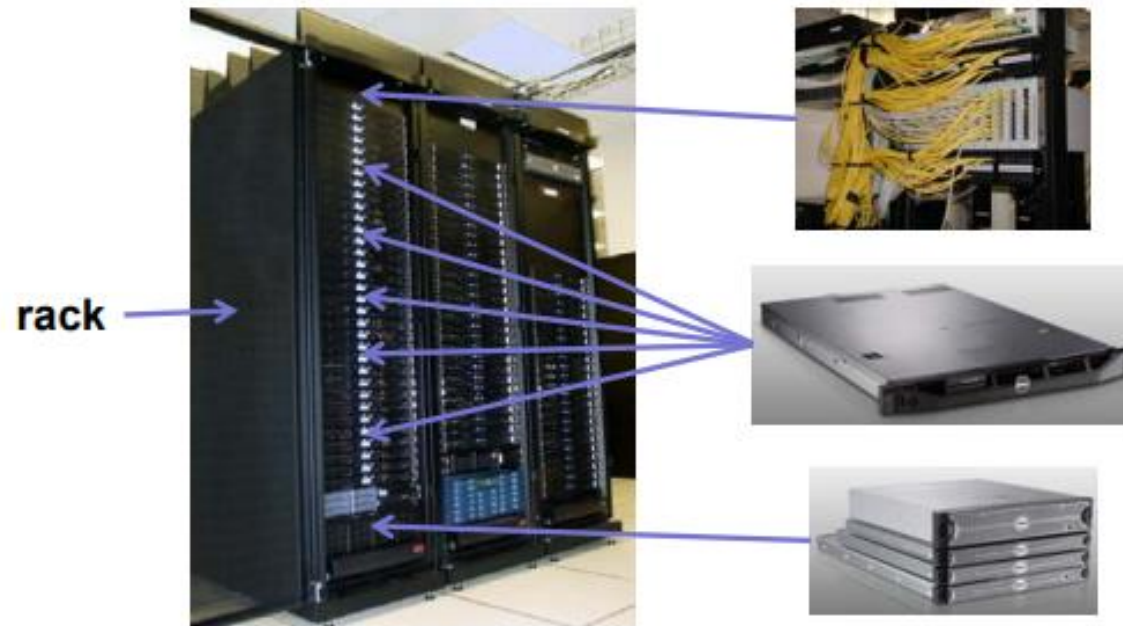
Cloud architecture





Enabling technologies of the cloud

Hardware:



network switches that connect servers to each other and with other racks

server/nodes/blade

storage devices

Enabling technologies of the cloud



Data center (co-location):



many racks in one
container



many containers in a
data center

data center





Enabling technologies of the cloud

Other technologies:

networks



electrical energy



conditioning



Software engineering / Laboratory Advanced Programming

Enabling technologies of the cloud



Many data centers:



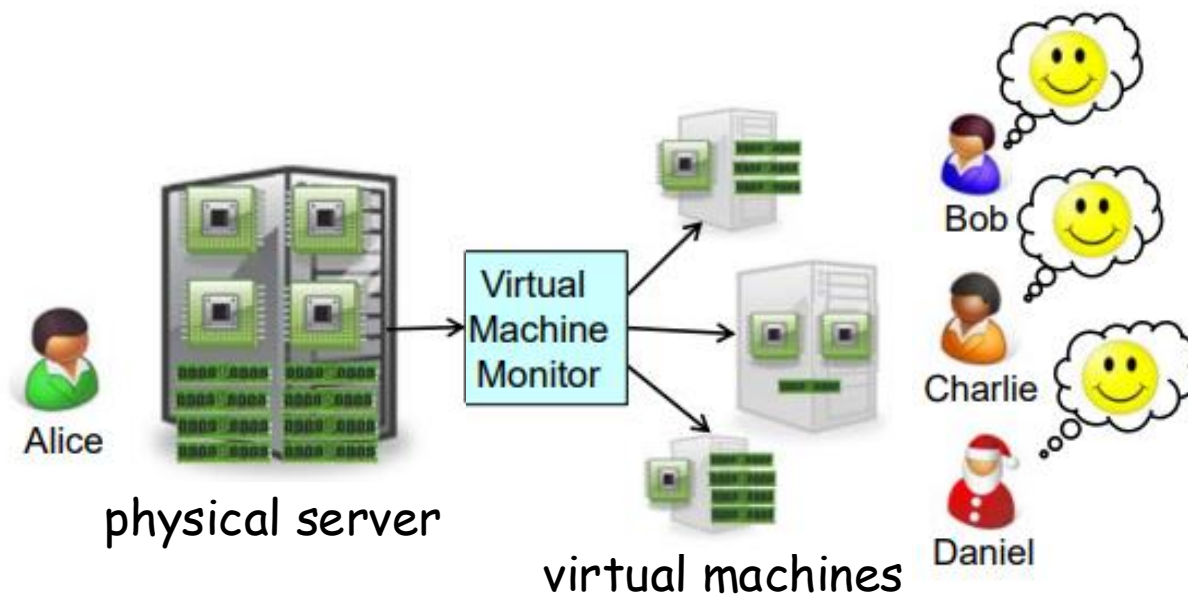
many data centers
in a **cloud**
infrastructure





Enabling technologies of the cloud

Virtualization - and tools to manage virtual environments:



Cloud computing and services



SAPIENZA
UNIVERSITÀ DI ROMA

In the cloud, a **service** is a well-defined **computational entity** managed by an organization, the service provider. It is **encapsulated**, meaning it has a contractually defined interface. Its implementation is **transparent** to users and can be accessed, through the Internet, by a client that is the user (or consumer) of the service.

It's a **high-level** definition.

Here are some examples of cloud computing services:





Google App Engine

Google App Engine (GAE) is a **PaaS**, a serverless platform fully managed by **Google Data Centers** to create highly scalable applications.

GAE supports several programming languages (*Java, PHP, Node.js and others*) offering flexibility in language choice.

Just write the code! GAE will run and manage the application in the **Google cloud**.

Characteristics:

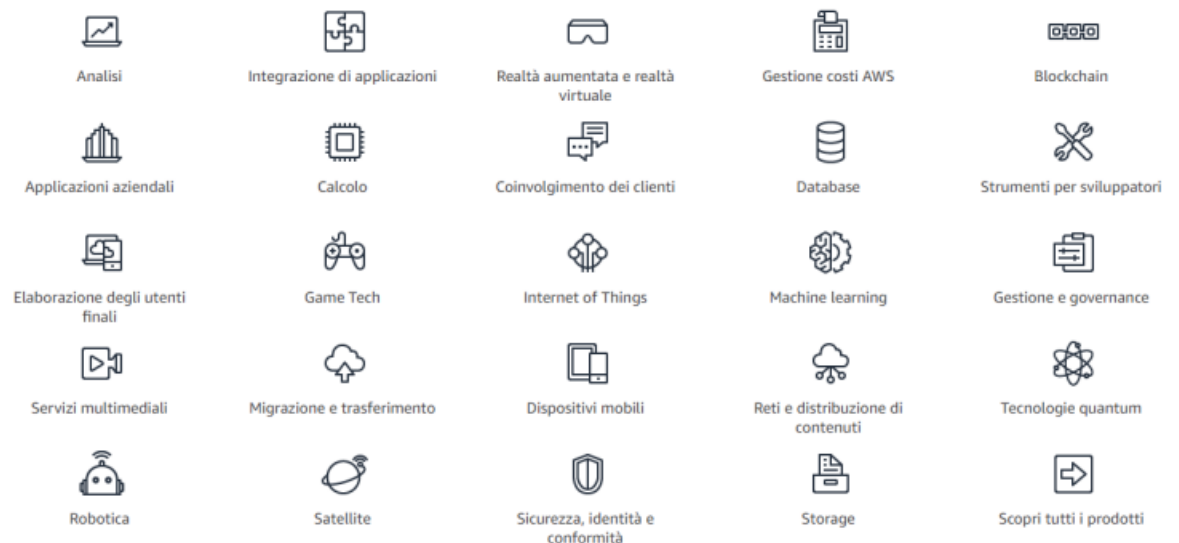
- automatic scalability and load balancing
- authentication and security (sandboxing)
- simplified administration
- initially free, then pay only what you use



Amazon AWS

Amazon Web Services (AWS) offers *compute services, database storage, content delivery, and other features* that are ideal for building sophisticated applications flexibly, scalably, and reliably.

AWS allows to build any application, regardless of business domain, allowing users to save time and money without compromising scalability or security.





Amazon AWS

AWS offers both Iaas and Paas services:

Infrastructure services (Iaas)

- *computing resources* - Amazon EC2 (virtual machines) and EC2 Container Service (container)
- *storage* - Amazon S3 and EBS
- *databases* - Amazon RDS and DynamoDB
- *networks* - Amazon Virtual Private Cloud and Elastic Load Balancing

Platform services (Paas) and application services

- *scalable web applications* - AWS Elastic Beanstalk and Lambda
- *messaging and publish-subscribe* - Amazon SQS and SNS
- *for developers (Dev) and operators (Ops)* - AWS CodeCommit, CodeDeploy and CodePipeline

Analytics, blockchain, IoT, machine learning, security, ...



Amazon EC2

Amazon Elastic Compute Cloud (Amazon EC2) allows users to run virtual machines on physical servers in the Amazon data center offering computing capacity in the cloud, **Amazon Machine Instance (AMI)**.

AMI represent virtual machine models and can be customized with a variety of OS and software. Amazon EC2 offers a wide range of predefined AMI types ranging from 1 to 96 VCPU 64-bit (based on *Intel Xeon processors*) and 0.5 GB to 192 GB of RAM.

AMIs can be preconfigured with a variety of operating systems (ex., *Linux or Windows*) and software (ex., *IBM DB2*) but, in the acquired AMIs, you can also install the software you want, including your own applications .



Amazon S3

Amazon Elastic Compute Cloud (Amazon EC2) is an *object storage service* offered by Amazon Web Services (AWS) in the cloud. Here are some features:

- It provides a simple web interface that supports both REST and SOAP, allowing users to store and recover their data easily.
- The stored data is represented as "objects". They can vary in size from 1 byte to 5 GB and are identified by a unique key.
- Users can perform **CRUD** (Create, Read, Update, Delete) operations on any number of objects stored in Amazon S3.



Amazon S3

- Objects can be stored both in Europe and the US, offering **flexibility** in data location.
- It offers various **security** features, including authentication, access control via ACL (Access Control List), and support for private and public objects.
- It offers different levels of **reliability** to ensure data availability and durability of stored objects.
- Can be used as a storage mechanism for **Amazon Elastic Compute Cloud (EC2)** instances



Amazon EBS

Amazon Elastic Block Store (Amazon EBS) is a block storage service. An EBS instance is represented as a block volume, which can be mounted on an Amazon Elastic Compute Cloud (EC2) instance.

An EBS volume can be used as a normal **hard disk** and can have capacities from 0.5 GB to several terabytes, offering flexibility in data management.

Amazon EBS offers different levels of performance and reliability, allowing users to choose the type of volume that best suits the needs of their application.

Users can select the availability zone where they want to allocate the EBS instance.



Amazon RDS e DynamoDB

Amazon Relational Database Service (Amazon RDS) is a web service that allows access, with all the features, to **relational databases** in the AWS Cloud (for example, *MySQL, Oracle, PostgreSQL or Amazon Aurora*).

Amazon DynamoDB is a fully managed service of **NoSQL** database, **non-relational database**, that offers high and predictable performance, optimal scalability and reliability.



Discussion on AWS IaaS services

AWS infrastructure services can be used in an integrated way to create execution environments, even complex ones, in which to release their applications.

These computing resources can be provisioned interactively:

- in an **interactive way**, via a web interface
- in an **automated way**, using scripts and other AWS support services

These services enable the creation of **multiple execution environments** and follow a **usage-based payment model**.

AWS offers a promise of availability of 99.99% per month and provides discounts if availability falls below certain levels, such as 99.0% (with a 30% discount)



Amazon Elastic Beanstalk

Elastic Beanstalk is a service that allows to quickly deploy and manage applications in AWS Cloud. It reduces the complexity of management not limiting choices or control.

It supports applications developed in *Go, Java, .NET, Node.js, PHP, Python and Ruby*.

Just load your application for Elastic Beanstalk to **automatically manage** all the details related to capacity provisioning, load balancing, sizing and application status monitoring.

There is no additional cost for Elastic Beanstalk: the payment is made for the AWS resources necessary for the storage and execution of its applications.



Amazon ECS

"**Amazon Elastic Container Service (Amazon ECS)** is a fully managed container orchestration service".

It is a highly **scalable service** (partly platform, partly infrastructure) that avoids having to manage (install and size) an infrastructure for managing a container cluster. It allows you to easily run container applications in a cluster, managed through a set of Amazon EC2 nodes.



AWS Lambda

"**AWS Lambda** is a processing service that allows you to run code without having to provision or manage servers".

It is a fully managed serverless processing service ("without server") that allows you to run application code (Lambda functions) in response to certain events, ex. HTTP requests via *Amazon API Gateway*, *S3 object changes* or *DynamoDB table updates*.

It automatically manages the computational resources needed to perform these functions.



Discussion on AWS Paas services

AWS platform services enable the development and release of applications on a standard architecture in scalable execution environments.

When you use a PaaS service, an execution environment based on IaaS resources is automatically created. IaaS and PaaS services offer abstractions at different levels, with PaaS services implemented through the composition of IaaS services.

The use of PaaS services implies a greater focus on the functionality of the application and the renunciation of control over the architecture and the execution environment.



AWS and DevOps

AWS also provides a set of services to quickly and reliably create and release software to the AWS cloud via **DevOps** practices, a combination of **development (Dev)** and **operations (Ops)**. For example:

- infrastructure provisioning and management
- code management of applications
- automation of the software release
- monitoring

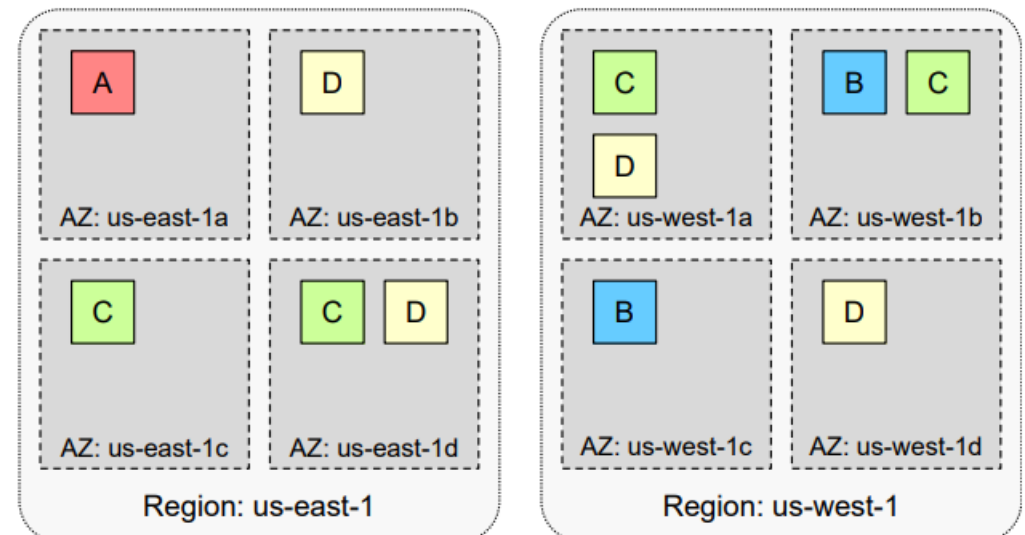


Regions and areas of availability

AWS cloud operates in more than 70 availability zones across more than 20 geographic regions worldwide.

A **region** is a geographical location, partitioned into **availability zones**, where AWS data centers are grouped together. For example, *EU (Ireland) (eu-west-1)* and *US East (Ohio) (us-east-2)*.

An Availability Zone (AZ) is a set of one or more data centers with redundant (independent) power, network and connectivity in an AWS region.

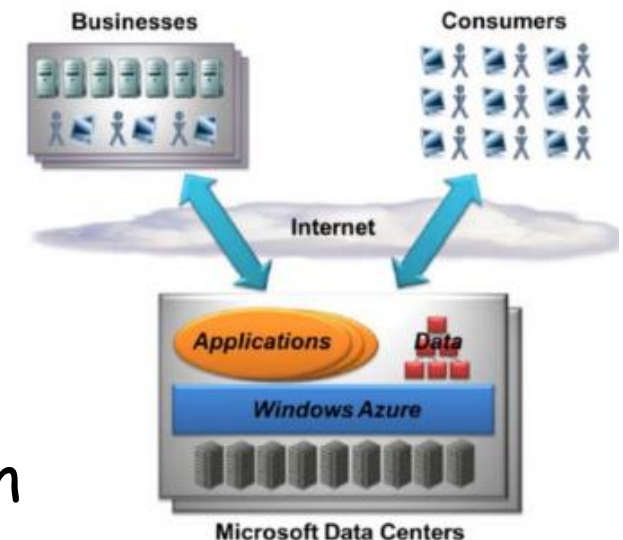




Microsoft Azure

Microsoft Azure is a platform that allows the use of resources in Microsoft cloud computing. It provides a range of **cloud services**, such as *analytics, compute, storage, and networking*, that can be chosen by users to develop and scale new applications or run existing applications in the cloud.

It is a **complete** and **flexible** platform that also provides numerous "open" solutions, not necessarily linked to the Microsoft world. For example, it supports Linux virtual machines and provides Docker container orchestration services.





Microsoft Azure

Some technologies of Microsoft Azure:

- **Virtual Machines** (Windows or Linux virtual machines), for running applications and storing data on computers in Microsoft data centers
- **Azure App Service**, for the development of advanced web applications, running in the Microsoft cloud, in a fully managed platform
- **Azure SQL Database**, a relational database manager based on SQL Server
- **Azure Kubernetes Service (AKS)**, container orchestration infrastructure services
- **Azure Functions**, event processing with serverless code
- **Visual Studio**, flexible environment for cloud application development



Microsoft Azure

Some Microsoft cloud data centers (expanding):

- in *Dublin*, a traditional data center - 38500 m² (2013)
- in *Chicago*, a container-based data center (2010) -112 containers, 1800-2500 container servers, 65000 m² (equal to 9 football fields)





Salesforce.com

Salesforce.com, one of the leading cloud computing platforms, is a leader in the **SaaS industry**.

It offers everything you need to manage your cloud business: applications (customizable), (for example, for *sales, customer support and collaboration*) accessible from different devices.

It also serves as a platform for custom application development (**PaaS**), giving companies the flexibility to adapt solutions to their unique needs.





Cloud Foundry

Cloud Foundry is an open-source cloud computing platform (**PaaS**). Its goal is to simplify and accelerate the development, testing, release and scalability of applications by offering the choice between different clouds, development frameworks and application services. It has a **container-based architecture**, allowing developers to run applications in any programming language in isolated environments.

The ability to run and move containers across cloud environments without changing applications, and the ability to be released to any cloud, public or private, adds portability to application development and management.



Netflix

Netflix (SaaS) is a software application that distributes movies, television series and other entertainment content via the Internet. It uses a microservices architecture, where independent components are released in containers in the Amazon Web Services (AWS) cloud.

Netflix Inc. is the company that provides the SaaS service, acting both as a **customer** service provider and as a **consumer** of cloud services by Amazon AWS to support its operations.

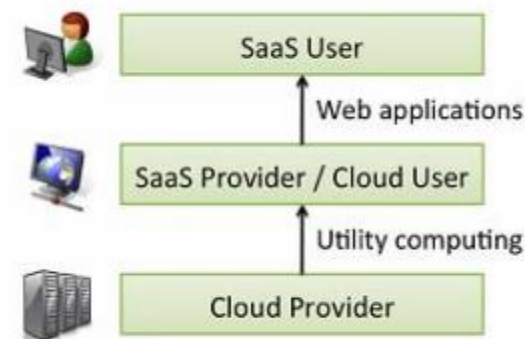
NETFLIX



Players involved

An organization or person may perform one (or more) of the following roles:

- **Cloud provider**, utility computing provider (IaaS or PaaS)
- **Cloud user**, user (consumer) of utility computing (IaaS or PaaS)
- **SaaS provider**, provider of a SaaS application. He could also be a cloud user
- **SaaS user**, user (consumer) of a SaaS application





Players involved

Let's take an example. We consider Mario Rossi, who is a client of the Netflix service, which is made by Netflix Inc. and released in the Amazon cloud.

- **Netflix** is a SaaS service
- **Mario Rossi** is a user (consumer) of the Netflix service (SaaS user)
- **Netflix Inc.** is the provider of the Netflix service (SaaS provider)
- **Amazon** is a provider of cloud services, IaaS and PaaS (cloud providers)
- **Netflix Inc.** is a consumer of Amazon's IaaS cloud services (cloud user)



Application fields

Some possible application fields of cloud computing:

- web applications
- desktop software extension (Matlab and Mathematica)
- applications with momentary need for large computing resources
- prototyping
- startup
- individual tasks
- carrying out research activities



Essential features

Five essential features for cloud computing:

- **Services on demand:** a consumer can acquire calculation resources unilaterally and automatically
- **Network access:** computing resources are accessible on the Internet
- **Resource pooling:** a provider's computing resources are brought together to serve many consumers, according to a multi-tenant model
- **Quick elasticity:** calculation resources can be obtained quickly and flexibly
- **Measurement of services:** the use of resources is controlled automatically, based on appropriate measures



Cloud computing

Cloud computing [Vaquero] is a large set of resources (such as hardware, development platforms and/or services) that are virtualized and easily accessible and usable.

These resources can be dynamically reconfigured, to adapt them to a variable load (they are scalable), favoring an optimal use of the resources.

They are typically used on the basis of a consumer payment model and come with guarantees, based on custom SLAs, offered by their supplier.



Cloud computing

Cloud computing [**Armbrust et al.**] refers to both applications delivered as Internet services and hardware and software in the data centers that provide these services.

- **Cloud** indicates the hardware and the software present in these data centers
- **Software as a Service** (SaaS) means applications delivered from a cloud
- **Utility computing** means the set of services provided by a cloud and made available to the public with payment methods to consumption
- **Cloud computing** is given by the union of SaaS and utility computing



Economics of cloud computing

From the point of view of the consumer, the aspects to consider in an economic evaluation of cloud computing are:

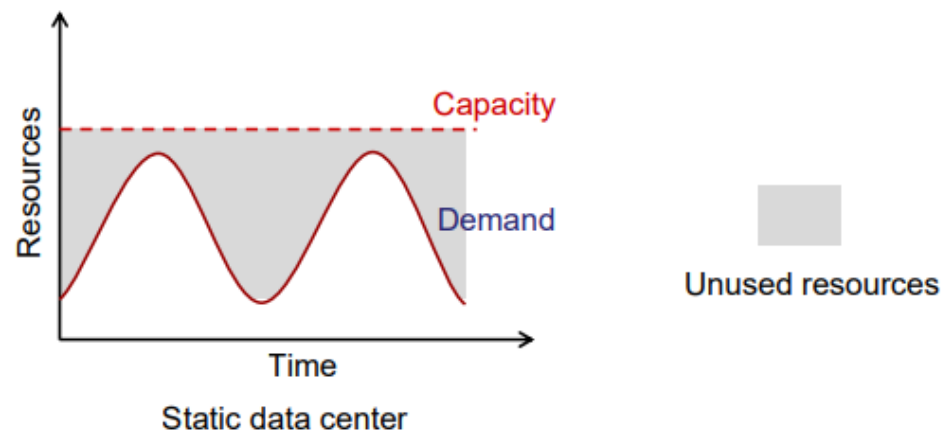
- **Consumer payment model**, allowing the transition from a capital expenditure system (CAPEX) to a current expenditure system (OPEX)
- **Elasticity**, which allows to mitigate the risks related to an incorrect sizing of the necessary infrastructure
- **Economies of scale**, in which suppliers can offer advantageous prices to consumers (and also advantageous for suppliers themselves)



Elasticity

Infrastructure sizing, when resource demand varies over time.

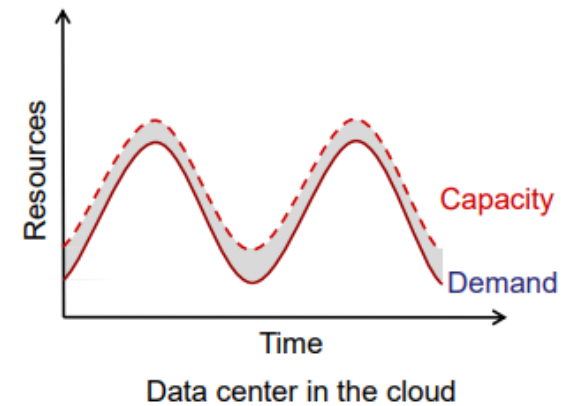
Provisioning for peak load



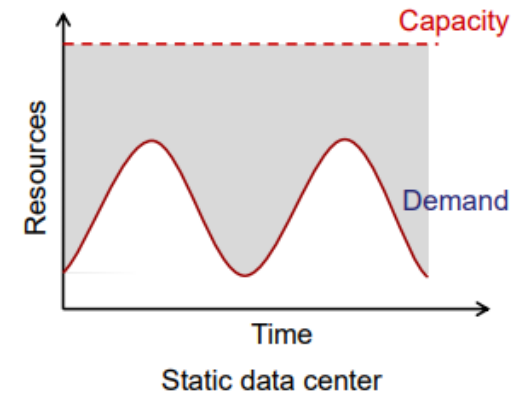


Elasticity

Cloud computing and elasticity



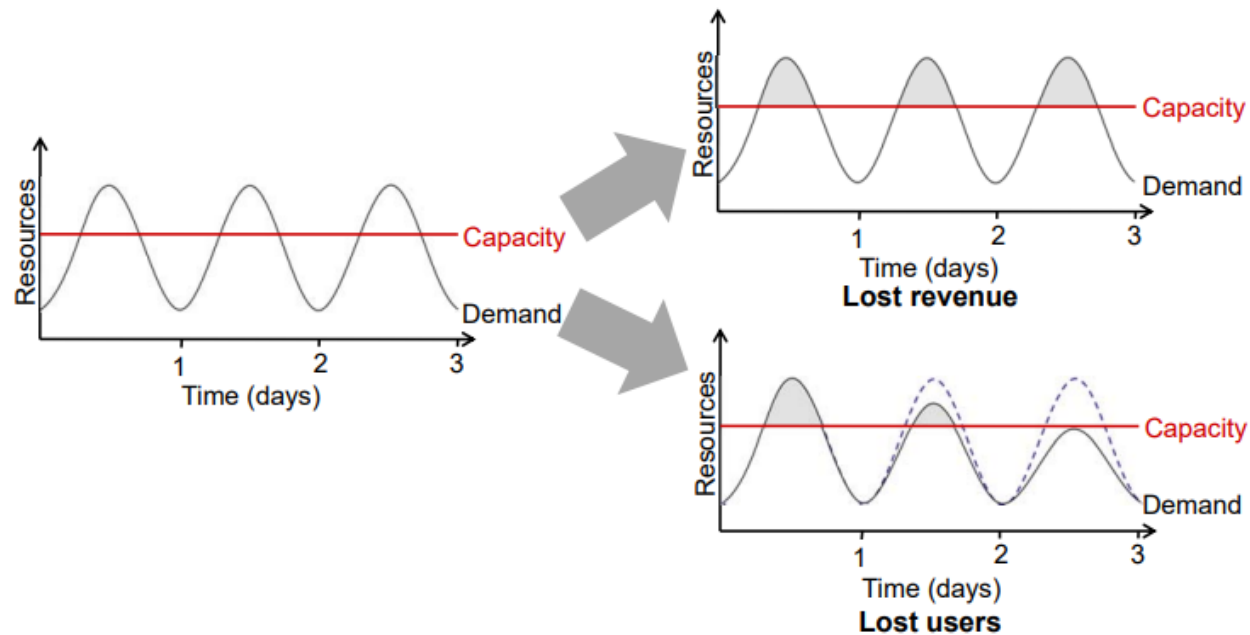
Over-provisioning





Elasticity

Under-provisioning





Economy of scale

A large data center can benefit from significant economies of scale compared to medium-small data centers, both for technological resources and for other infrastructures (electricity, heating, premises, ...).

It can sell these resources at lower costs than typical for a small to medium data center.

Risorsa	Costo per un DC medio	Costo per un DC molto grande	Rapporto
Rete	95\$ / Mbps / mese	13\$ / Mbps / mese	7.1x
Storage	2.20\$ / GB / mese	0.40\$ / GB / mese	5.7x
Amministrazione	≈140 server/admin	>1000 server/admin	7.1x



Economics of cloud computing

From the supplier's point of view, the possible benefits are:

- Profit by exploiting **economies of scale**
- **Capitalise:**
 - Amazon, exploiting residual computing capacity (outside peak periods)
 - Google, leveraging existing infrastructure
- **Defend a brand**
 - Microsoft, e.g. to sell tools .NET
- **Strengthen customer relationships**, e.g. by offering their customers a cloud disaster recovery service



Discussion

Some economic consequences of cloud computing:

- **Cloud service providers** can build very large data centers, and sell computational resources and services profitably
- **Cloud service users** can use or build software (scalable and available) on the basis of a consumer payment model, without initial costs and without additional infrastructure management costs
- The **cloud** also supports innovation and competition
- **Small software manufacturers** can start their businesses with less reliance on external investors
- **Investors** can target their investments more effectively



Software systems for the cloud

It is increasingly common today to release their software systems to the cloud. This raises a set of risks, opportunities and challenges.

Risks -> include releasing into a shared execution environment and using "non-traditional" cloud platforms and services.

Opportunity -> it is the release into elastic, scalable and available platforms.

Challenge -> it is to effectively make an application or service scalable, available and editable.

Cloud software requires an architectural change that must take into account the characteristics of the cloud.



Software systems for the cloud

Some **requirements** for cloud software systems:

- High availability: no service interruption
- Scalability: accept a growing number of users or requests
- Modifiability: rapid (continuous) development cycles and feedback
- Support for mobile clients and multiple access devices
- IoT support (Internet of Things)
- Support for Big Data



Software systems for the cloud

Some **cloud features** you need to consider:

- Use of shared resources and limited control over their location
- Continuous changes in the execution environment.
- It is generally necessary to use the specific services offered by the cloud provider, which may be different from those used traditionally, and also automate the management of infrastructure and releases
- It is often necessary to apply specific tactics and patterns



Discussion

It is useful to discuss cloud computing in the specific context of software delivery.

The cloud offers several software-oriented service models, providing infrastructure resources (IaaS) and platforms (PaaS), which support different options for execution environments.

Moreover, the cloud allows different deployment models the cloud also provides **DevOps services**, to manage software release in an automated way.



Cloud computing virtualization and containerization



Virtualization based on containers

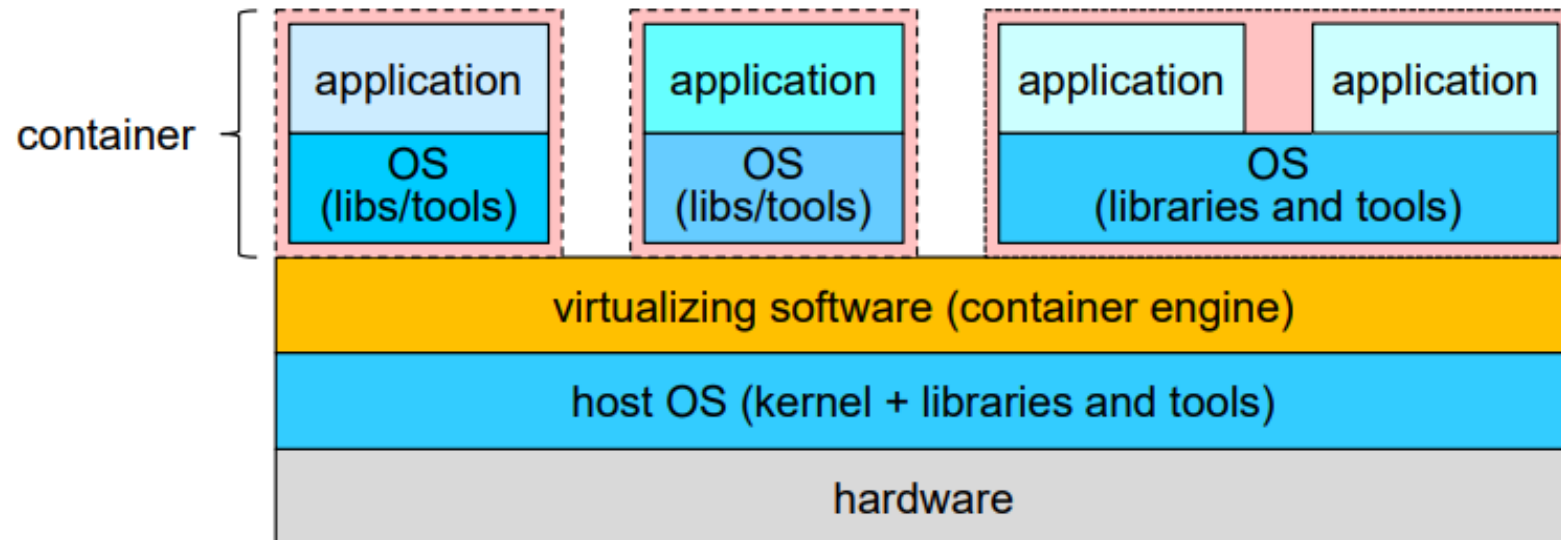
Container-based virtualization (containerization) is an infrastructure that allows applications to run in isolated environments known as **containers**.

Containers are a software distribution unit that encloses the application and all its dependencies (libraries, tools, configuration files, etc.) in an isolated environment.

These containers are known to be **lightweight** and quick to boot, as they share the host OS kernel rather than virtualizing an entire operating system.



Virtualization based on containers





Virtualization based on containers

Container-based virtualization, especially in the Unix/Linux world, is a form of lightweight virtualization, also called OS level virtualization because it is supported directly by the host OS kernel.

Container virtualization software (**container engine**) allows to define a container as a set of processes and other resources. For example, a container running a Java service essentially consists of a JVM process (running on the host).



Virtualization based on containers

We can make some **considerations** about container-based virtualization:

- The host system runs only one shared kernel, which manages both the resources of the host system and those of the running containers.
- The kernel is shared by containers, each of which runs its own OS and has its own "virtual" resources (for example, its own complete file system and its own network stack, with its own IP address).
- Container virtualization software ensures isolation between containers and between containers and host.



Virtualization based on containers

- The shared kernel handles all processes and all other resources (ex., files), both of the host and containers
 - processes in a container are managed as host processes
 - the file system of a container is managed as a sub-tree of the host file system (For example, from `/var/lib/docker/containers/containerid/filesystem/`)
 - access to a file in a container is realized as access to a file in the host file system



Container

A **container** is a standardized software unit that bundles one or more software applications, along with their configurations and dependencies, so that these applications can run quickly and reliably in an appropriate container execution environment.

Each container (container instance) is typically used for run a specific software service or application, that is to realize to realize a complete and autonomous virtual execution environment, with all the necessary dependencies (the OS libraries and tools, the runtime libraries required by the languages used and the middleware).



Container

The container is also a **standardized execution environment** for its application (or applications). In this way, each application (in its own container) can be released and run consistently on a variety of platforms.

Containers introduce a **lower overhead** compared to VMs (they are "lighter"): performance is almost native, meaning you don't have to use processor virtualization or I/O virtualization or a hypervisor.

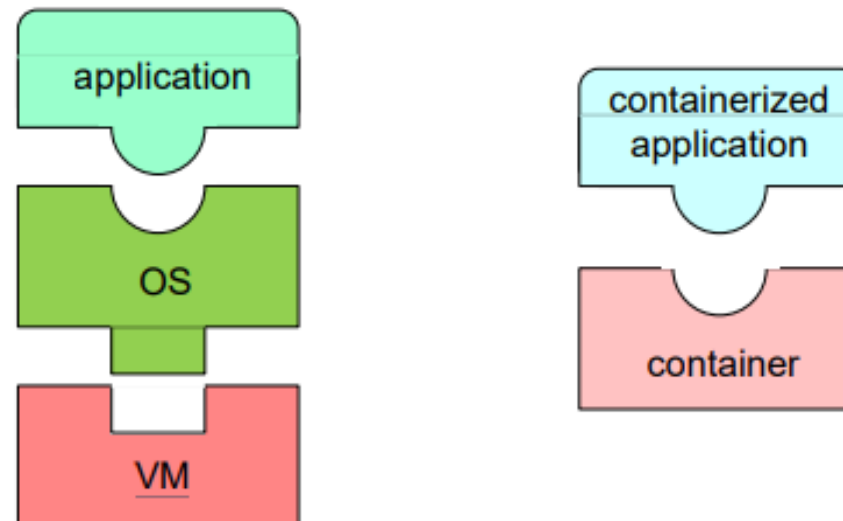
Containers, however, offer less operational flexibility and isolation than VMs: the OS of a container must be compatible with the kernel running on the host.



Container and interface

Let's compare VMs and containers from the **interface point of view**:

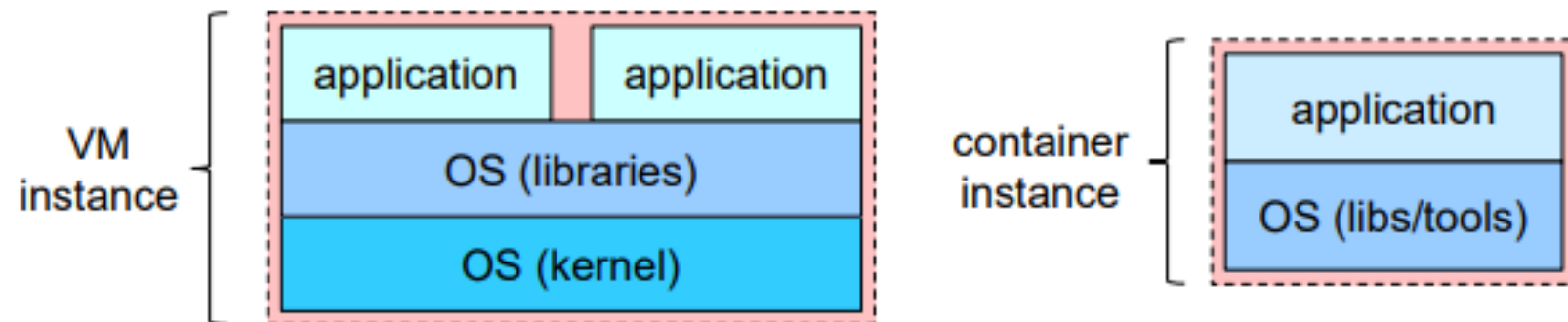
- the interface exposed by a VM is that of the hardware of a computer
- the interface exposed by a container is that of the kernel of an OS, the interface of the kernel system calls





Container and instance

Similar to a VM instance, a container instance also includes OS libraries and tools and applications that are installed there.





Container and images

A **container image** consists of the image of the file system of a container.

Similar to a VM image, it includes one or more applications to run in the container, along with the libraries and tools of an OS and all the software needed to run those applications, so that you can easily create one or more container instances from that image.



Types of containers

Here is a classification of containers:

- An **OS container** is a container designed to be used as a lightweight VM, with its own OS to run multiple applications or services.
- An **application container** is a container designed to hold and run a single application or service.



Application container

Application containers allow to focus each container on a single application or service:

- **Application Isolation:** Containers allow to isolate individual applications or services within an autonomous environment. This leads to reliability and safety.
- **Dependency Management:** Each container contains only the dependencies needed to run the specific application or service. This reduces the risk of conflicts or between dependencies of different applications.
- **Lightness and Speed:** Containers are lightweight and can be created and started quickly. They only use the resources needed to run the specific service, which improves efficiency and scalability.



Techniques for container-based virtualization

There are several container technologies in the context of UNIX and Linux operating systems. For example:

- **LXC**
- **OpenVZ** for Linux
- **Solaris Containers** for Solaris
- **FreeBSD jail** for FreeBSD
- **Docker**



Container LXC

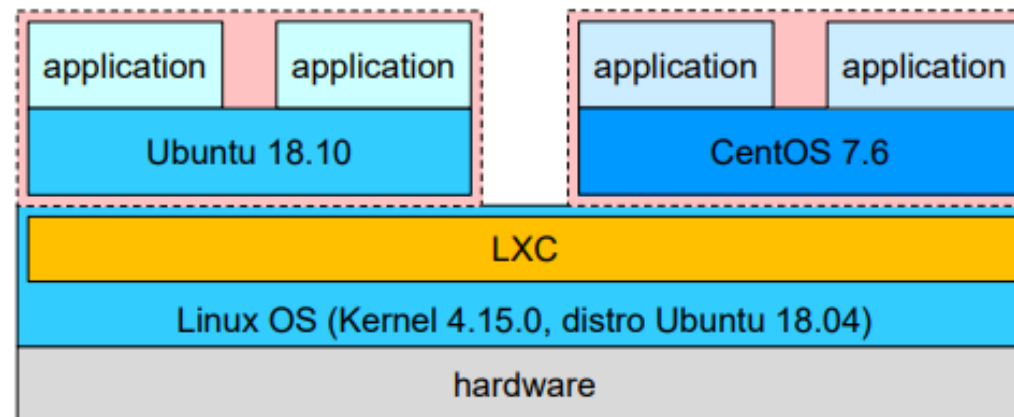
LXC (Linux Containers) is a set of simple but powerful API-based tools to create and manage containers on a Linux host (real or virtual).

In the Linux kernel, each process can generate child processes hierarchically, creating a system process tree. Each container is a separate part of this hierarchy and has its own associated resources such as *CPU*, *memory*, and *disk space*.



Container LXC

While the kernel used by an LXC container is that of the **Linux** host, it is possible to run a different operating system inside a container. This means that you can have containers running different Linux distributions or different versions of the same Linux operating system.





Container LXC

LXC lets to **create** and **run** one or more containers, along with their applications. This is possible because an LXC container is isolated from other containers and the host OS, preventing container processes from directly interfering with processes in other containers or the host.

LXC containers provide an **isolation** level similar to virtual machines (VMs), but without the need for a hypervisor.



Container LXC

LXC combines some Linux kernel features to provide container abstraction: **control groups (cgroup)** and **namespaces**.

The first is a group of processes, comprising the root process and all its current and future children (and descendants), to which parameters are associated (such as CPU, memory, network, file system, ...).

Cgroups allow you to isolate, limit, and measure the use of resources assigned to a group of processes.



Container LXC

A **namespace** represents a self-contained collection of resources to which virtual names are given, which are then mapped to real resources (for example, *process and user IDs, network resources, host name and ports, files in the file system*).

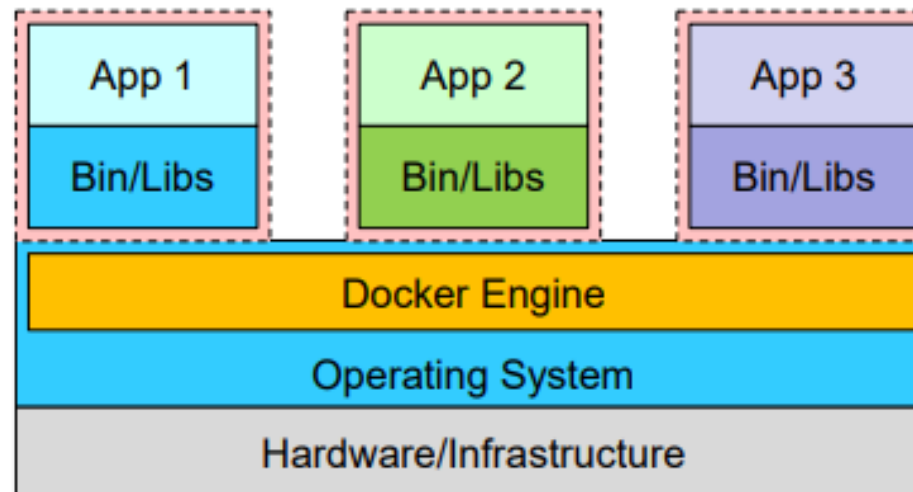
The namespaces allow to decouple a group of processes from the real resources that will be assigned to them, to control the visibility of resources and to avoid name conflicts and inconsistencies in references.



Container Docker

Docker is a container platform to build, release and run distributed applications in a simple, fast, scalable and portable way.

It is light, standardized and open and safe.





Container Docker

Below, the main features offered by the Docker platform:

- create a container (a container instance) from a container image
- initiate, monitor, inspect, arrest and destroy containers
- create and manage container images
- manage related groups of containers, in which to run multi-container distributed applications



Container and virtual machines

Containers offer **less flexibility** than VMs: while VMs have their own full OS and applications, the OS of a container must be compatible with the host OS (usually Unix or Linux).

In addition, the flexibility offered by VMs comes at a cost, since it requires a larger amount of resources in the host system, can introduce more execution overhead, and takes longer to boot. Containers, on the other hand, are **"lighter"** than VMs. In fact, they require fewer resources, performance is almost native, and containers can be created and started faster.

From the standpoint of isolation, while between containers it turns out to be **not complete**, between VMs it is.



Container and software release

Containers are another release option for **distributed software systems**: each container ("application container") encapsulates a software service, along with the software stack needed for that service.

Here are the **benefits** of this choice:

- each container encapsulates a single software service, easily and reliably
- fault isolation and safety
- the containers are light
- containers can be released both in the cloud and on premises and, in part, in a platform for container orchestration (e.g., **Kubernetes**)



Container and software release

Below, the **drawbacks** of using containers for software release:

- the isolation between containers is **not complete**
- **overhead in container images** administration and update
- **overhead in container execution** infrastructure administration, unless containers run in a cloud hosted solution (such as **AWS ECS** or **Google GCE**)



Docker

Docker (www.docker.com) is a container platform for building, releasing and running distributed applications, in a simple, fast, scalable and portable way.

A Docker container is a **standardized software** unit that bundles a software service, along with its configurations and dependencies.

Docker containers are lightweight (they use few resources and start quickly), standardized and open (and therefore portable: they can run with major Linux distributions and with Windows and Mac OS, and even in the cloud) and secure.

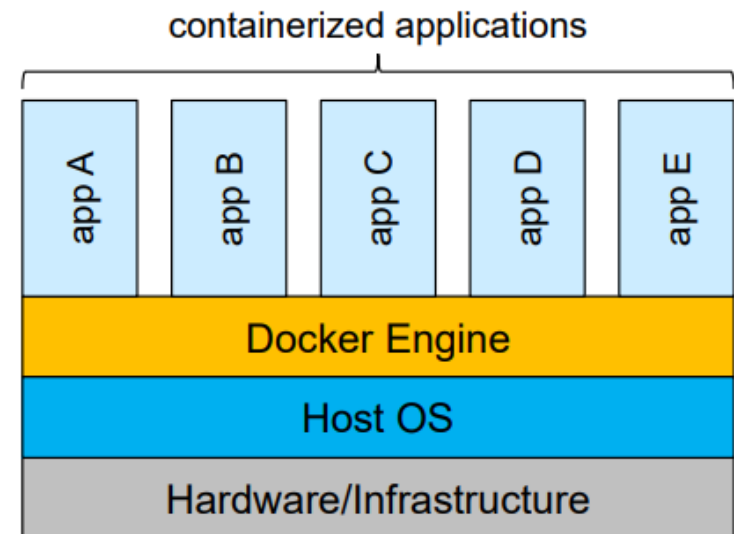


Docker

The Docker platform (2013) was initially built over LXC containers (2008).

Today it is based on **containerd** and **runc** libraries (2014, 2015), as well as **cgroup** and **namespace**.

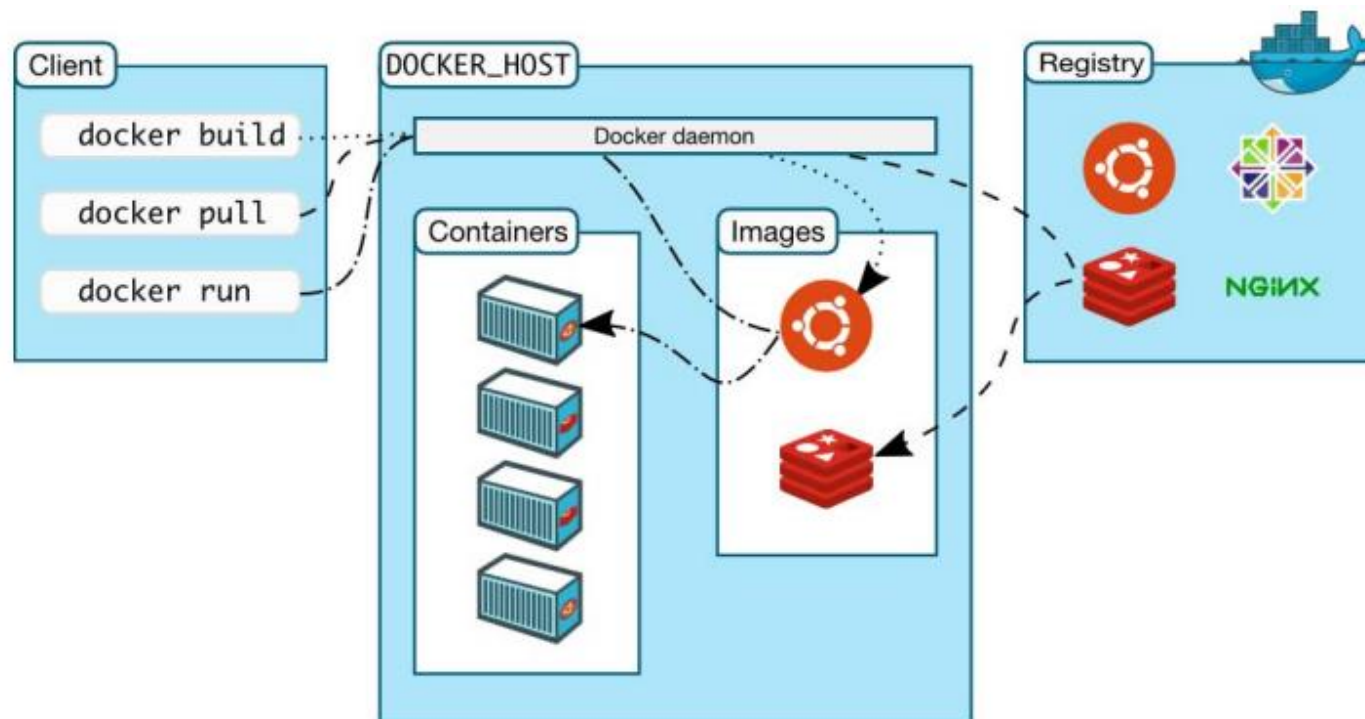
The Docker platform enables separation between applications and the execution infrastructure to simplify application release and ensure the portability of services deployed through containers.





Docker Engine

The core of the Docker platform is **Docker Engine**





Docker Engine

Docker Engine is based on a client-server architecture:

- The **server** is a host that can run and manage Docker containers: it runs the **Docker daemon** process (**dockerd**) and manages a set of Docker objects (containers, images, networks and volumes).
- The **client** (**docker**) accepts commands from the user via a CLI interface and communicates with the Docker daemon on the host via a REST API.
- The **registry** contains a set of images. Docker's public registry is **Docker Hub**.



Image registry

A **registry** is a service (public or private) that contains a collection of container images. A public registry generally hosts basic images, which may only include an operating system or, in some cases, include basic software (examples of basic images are *ubuntu*, *postgres*, *wurstmeister/kafka* and *openjdk*).

Docker Hub (<https://hub.docker.com>) is the public registry of Docker.

A **repository** is a portion of a registry that contains a set of container images, usually variants or different versions of the same image.



Docker objects

A **container** is an executable instance of a container, created from a Docker image, that contains an application or service, along with everything you need to run it.

This is a **dynamic entity** that exists during execution (runtime).

It can run on a host and represents the executable drive. has its own state, which can change during execution (e.g., file system content (on disk) or session status (in central memory)).



Docker objects

An **image** is a set of files that represent the snapshot of the file system of a container, it is a model for creating containers (eg, an image with an Ubuntu OS, Open JDK and a specific Java application of interest).

It is a **static concept**, which is not directly executed.

It has **no state** of its own, it is immutable.



Docker

Interaction with a Docker host occurs through an interface that can be the **Command-Line Interface (CLI)** or the remote interface based on a **REST API**.

This API is based on the **docker command**, which offers numerous options, commands, and operations for managing images, containers, and other Docker objects, allowing to manage their lifecycle.

Docker image commands are used for image management, while **docker containers** are used for container management. These commands provide functionality for creating, starting, stopping, removing, and managing the status of Docker objects.



Docker commands

- **docker image build** (or **docker build**) lets you build an image (custom)
 - `docker build -t image-name context`
- **docker container create** (or **docker create**) allows you to create a new container from an image
 - `docker create --name=container-name image-name`
- **docker container start** (or **docker start**) allows you to run a container (already created)
 - `docker start container-name`
- **docker container run** (or **docker run**) creates and runs a new container (possibly anonymous), using a single command
 - `docker run [-name=container-name] image-name`



Docker commands

The command ***docker run hello-world*** is a classic example to start a Docker container using the **hello-world image** available in the Docker Hub. This command is often used as an initial test to verify that the Docker installation is working properly and can run containers.

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

```
To generate this message, Docker took the following steps:
```

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

```
To try something more ambitious, you can run an Ubuntu container with:
```

```
$ docker run -it ubuntu bash
```

```
Share images, automate workflows, and more with a free Docker ID:
```

```
https://hub.docker.com/
```

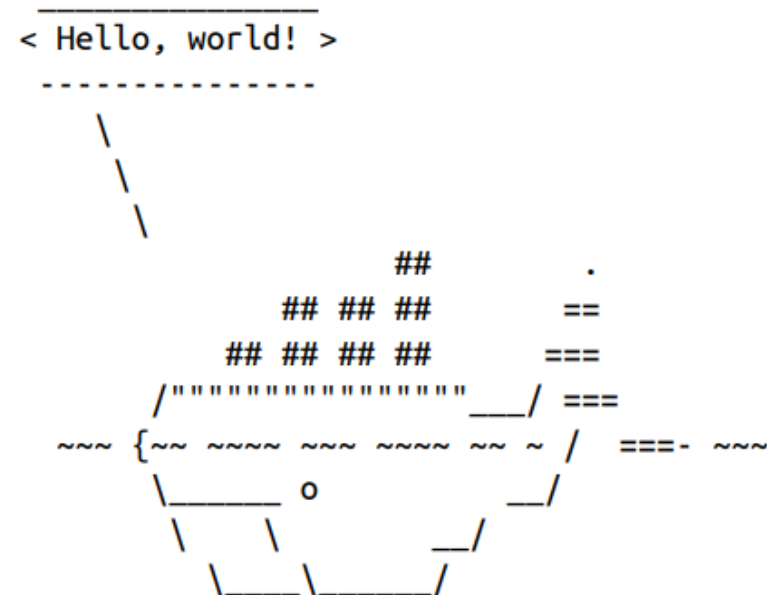
```
For more examples and ideas, visit:
```

```
https://docs.docker.com/engine/userguide/
```


Docker commands

The command **docker run docker/whalesay cowsay Hello, world!** starts a Docker container using the **docker/whalesay** image, which displays a whale, using the **cowsay** program inside the container. The word "Hello, world!" is passed to cowsay to be shown by the whale as if it were her message.

It is an easy way to check the basic Docker configuration and connectivity to the image log.





Imaging

To create a custom image, Docker uses a "**infrastructure-as-code**" approach based on a special text file called **Dockerfile**. This file contains a set of commands that Docker runs to build a custom image.

The command **docker build -t image-name context** allows the automatic construction of an image named image-name, using a specified "context". The context can be a local folder or a location within a Git repository.

The context must include the Dockerfile and all other files needed to create the custom image, such as binary files, scripts, and templates, providing Docker with the essential resources to create the desired image.



Dockerfile

A **Dockerfile** consists of a sequence of instructions

```
# Hello world  
FROM busybox:latest  
ENTRYPOINT ["echo", "Hello, world!"]
```

FROM specifies the base image from which to build the custom image (e.g., *busybox* (is a minimal Linux distribution) or *ubuntu:18.04*).

ENTRYPOINT ["executable", "param1", "param2", ...] is an instruction that specifies the executable or command that must be executed by the containers that will be created by this image.



Dockerfile - CMD

The **CMD** statement allows to specify arguments for the **ENTRYPOINT** statement, with the observation that these arguments can be overwritten when the container starts.

```
# Hello world
FROM bosybox:latest
ENTRYPOINT ["echo"]
CMD ["Hello, world!"]
```

- `docker build -t myhello2 .`

- `docker run myhello2`

Hello, world!

- `docker run myhello2 Ciao, mondo!`

Ciao, mondo!



The RUN instruction

The **RUN** statement within a Dockerfile specifies a command that must be executed during the construction of a Docker image. This command is used to perform actions when creating the image itself, such as installing dependencies, configuring the environment, and running scripts.

In contrast, the **ENTRYPOINT** statement defines the main command that will run when you start a container based on the built image. This command is the container's default action and can be overwritten when the container is running, adding additional commands or parameters.



The **VOLUME** instruction

The **VOLUME** statement defines a mount point for data from the host system or another container. Should be used in conjunction with other options of the **docker create** and **docker run** commands.

The **-v host-src:container-dest** option is used to mount a specific folder from the host system inside the container, allowing data sharing between the host and the container.

-volumes-from=container-name allows to mount a volume inside a container that is handled by another container. This allows to share volumes between containers without having to repeat the volume definition in each Dockerfile.



The EXPOSE instruction

The **EXPOSE** statement specifies the ports on which the container is listening during execution. You must use other options when creating or running the container to make port forwarding.

The **-p *host-port:container-port*** option is used to publish a specific port exposed by the container to a specific port on the host. For example, **-p 8080:80** would instruct Docker to map port 80 of the container to port 8080 of the host.

The **-P** option publishes all ports expressed by the container on random ports of the host.



Apache HTTP Server

Using Dockerfile, it is possible create an image that starts an Apache HTTP server inside a Docker container when it runs.

```
# Dockerfile for Apache HTTP Server
```

```
FROM ubuntu:18.04
```

```
# Install apache2 package
```

```
RUN apt-get update && \  
    apt-get install -y apache2
```

```
# Other instructions
```

```
ENV APACHE_LOG_DIR /var/log/apache2
```

```
VOLUME /var/www/html
```

```
EXPOSE 80
```

```
# Launch apache2 server in the foreground
```

```
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```




Apache HTTP Server

- **FROM:** specifies the base image as Ubuntu 18.04.
- **RUN:** updates the repositories and install the apache2 package inside the image.
- **ENV APACHE_LOG_DIR:** sets an environment variable for the Apache registry path.
- **VOLUME:** declares a volume for the directory /var/www/html, which is the default directory where Apache stores the website files.
- **EXPOSE 80:** Exposes port 80 to allow connection to Apache HTTP server via HTTP protocol.
- **ENTRYPOINT:** sets the entrypoint to start Apache in "FOREGROUND" mode, allowing the server to execute foreground requests.



Apache HTTP Server

- **Construction of the image:** `docker build -t myapache .`
- **Creation of the container:** `docker create -v ~/projects/www:/var/www/html -p 8080:80 --name=myapache myapache.`

The pages served by the HTTP server are those in the host directory `~/projects/www`.

The HTTP server is redirected to port 8080 of the host.

- **Execution of the container:** `docker start myapache`
(start the myapache container)

It is possible to access the HTTP server from the host on `http://localhost:8080`.



Docker commands

- To list running (or even stopped) containers: *docker container ls* or *docker ps [-a]*.
- To inspect used ports from a container: *docker container port container-name* or *docker port*.
The result is *80/tcp -> 0.0.0:8080*
- To inspect a container or image: *docker container inspect container-name* or *docker inspect*.
Returns container or image information (in JSON format).



Docker commands

- To view generated logs in a container: *docker container logs container-name* or *docker logs*.
- To stop a container running: *docker container stop container-name* or *docker stop*.
- To remove a container: *docker container rm container-name* or *docker rm*.
- To stop all running containers (use with caution!): *docker stop \$(docker ps -a -q)*.
- To remove all containers (use with caution!): *docker rm \$(docker ps -a -q)*.



Docker commands

- To list images in the local cache: *docker image ls* -
oppure docker images.
- To remove an image from the local cache: *docker*
image rm image-name or *docker rmi*.
- To remove all images from the local cache (use with
caution!): *docker rmi -f \$(docker images -q)*.



Docker commands

- Docker client can also be used to specify commands to run on a remote Docker host *docker-host*:
 - The *docker -H=tcp://docker-host:2375* command explicitly specifies the remote Docker host and the TCP protocol with port 2375 to run a Docker command on that host.
 - *docker -H=tcp://docker-host:2376* command is similar to the previous one, but uses port 2376 which supports secure access via TLS.
- Alternatively, you can specify the remote Docker host using the **DOCKER_HOST** environment variable:
 - *export DOCKER_HOST=tcp://docker-host:2375*. All the Docker commands that are executed will be routed to this host instead of the local host.



Image (and container) format

The **file system** of a Docker image (or container) consists of a series of layers that contain files and directories.

They are combined into a single coherent view thanks to the **Union File System (UFS)**.

It is a **lightweight format** that allows sharing layers between similar images, reducing the disk space needed to store different images that share the same basic layers.

The top layer of a file system can overwrite or hide files of the underlying layers, providing a virtual union of all layers to create the final image or container state.



Image (and container) format

When a file is requested inside a container, it is read from the highest layer it is in. If the file is not present in that layer, Docker proceeds to look for it in the layers below in the order in which they were superimposed.

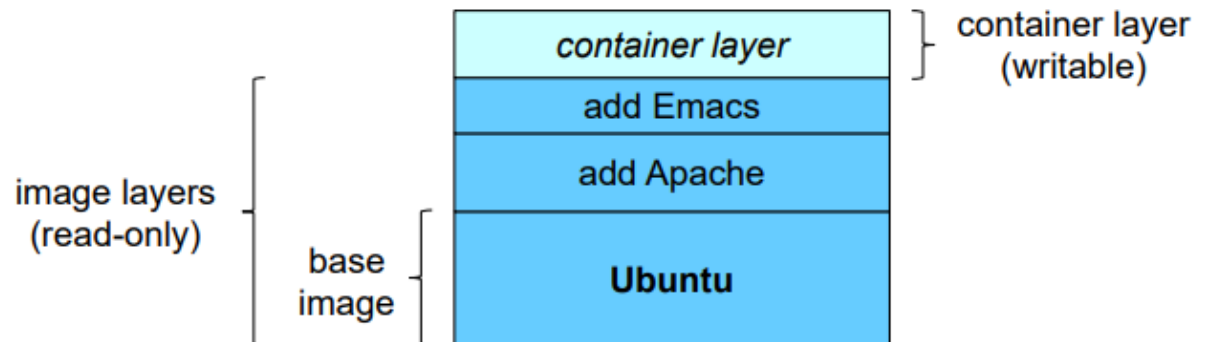
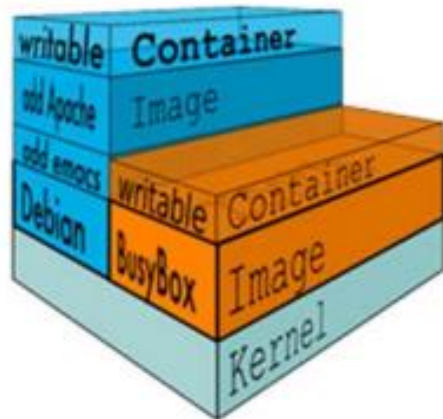




Image (and container) format

Each Docker image is based on a basic image, which usually includes an operating system and its libraries. This basic image is the first layer of the file system.

The next layers of the image usually correspond to the installation of packages, middleware or applications. Any changes made to an image are added as a separate layer, so that changes do not directly affect the previous layers.

Each container has an additional final layer, known as "**read-write layer**" or "**write layer**", that is the only editable part of the container's file system, preserving the previous layers of the image.



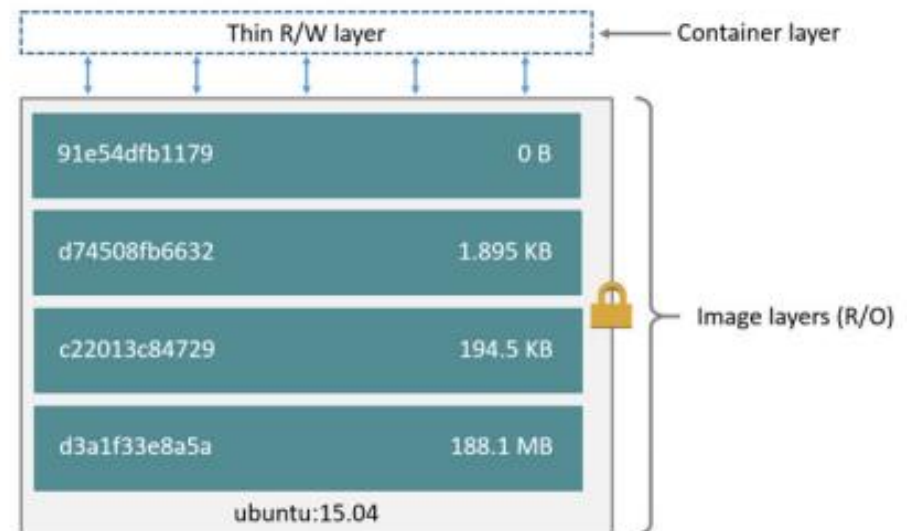
Image (and container) format

Image



Image

Container (File system)



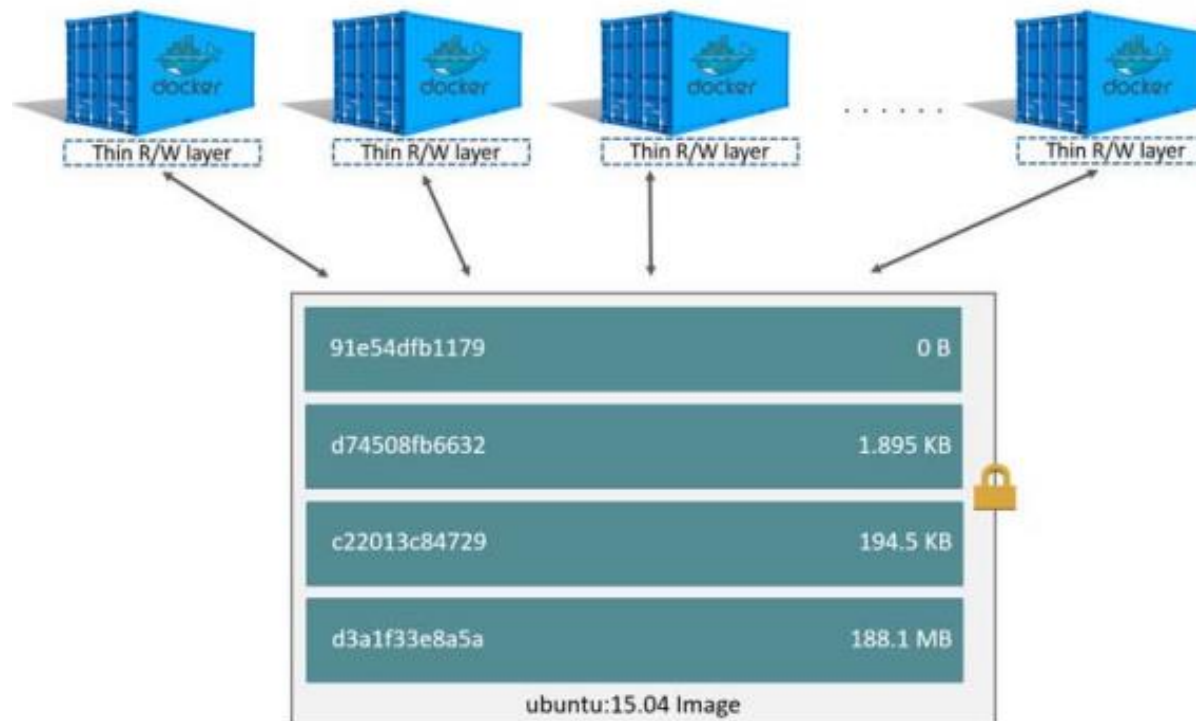
Container
(based on ubuntu:15.04 image)

Image (and container) format



SAPIENZA
UNIVERSITÀ DI ROMA

An image shared by multiple containers





Imaging

Building a custom **image** is based on running a Dockerfile, and happens as follows:

The base image specified by **FROM** is downloaded from the registry into an image cache on the host (if it is not already cached). This basic image (consisting of one or more layers) is used as the base layer of the new custom image.

Each Dockerfile statement (specifically, **RUN**) is performed using a new writable layer above the current image.

The result of executing a Dockerfile statement is then saved (before executing the next statement).



Creation of containers

Creating a container in Docker is always done from an image, which serves as the basis for the container's file system and its metadata.

The container file system consists of layers of the initial image to which a writable layer is added. The latter is specific to the container and allows the container to make changes, additions or cancellations during its execution. This writable layer is allocated to the host file system.



Execution of containers

Running a container in Docker involves several stages that prepare the environment and actually start the container:

Allocation of runtime resources: when a container is requested to run, the container engine allocates the runtime resources for the container. For example, it allocates (in the host kernel) a set of namespaces.

Network configuration: after namespace allocation, the container engine configures the network for the container. This may involve creating a new virtual network interface for the container or associating the container with an existing network, depending on the configuration specified.



Execution of containers

Starting the container: once the namespaces and the network are configured, the container engine starts the container using the specified image. This boot involves creating an image-based container instance with its file system and initial configuration.

Executing the command: finally, once started, the container executes the command specified by the **ENTRYPOINT** in the Docker image, which defines what the container will do during its execution, for example, launch an application, a service, or any other specific operation.



Volumes and data sharing

A **volume** is a directory outside the container **Union File System (UFS)**.

A volume can be accessed, shared and reused by multiple containers.

A volume allows to manage persistent data, regardless of the life cycle of the individual containers that can access it.



Volumes and data sharing

The `-v` option of *Docker create* or *Docker run* allows to associate a directory or file system of the host with a directory within the container.

Using the `-v` option followed by the host location and container location specification, it is possible to mount a host folder inside the container. For example, `docker run -v ~/projects/www:/var/www/html ...` associates the host's `~/projects/www` folder with the `/var/www/html` path inside the container.

When using this mount type, the data in the host folder (in this example, `~/projects/www`) is also accessible from the container.



Volumes and data sharing

It is possible to have **shared volumes** between containers but not linked to a specific host folder. In this case, the ***-volumes-from*** option must be used.

A container must first be created, inside which the volume will reside.

Then it is possible to create other containers that access that shared volume, with the ***-volumes-from container-name*** option. If the container in which a volume resides is deleted, the volume is still retained (unless explicitly cancelled).



Networks

Docker offers advanced network management capabilities to enable **communication** between containers and with the host.

When installed, Docker automatically creates three networks:

- **Bridge**: uses the docker0 virtual interface on the host. The IP addresses assigned to this network are in the 172.17.0.1/16 range.
- **Host**: connects a container directly to the host network, sharing the same network and network stack as the host.
- **None**: removes the network, isolating the container from network traffic.



Networks

When a container is started, Docker automatically assigns a free IP address from the "bridge" network subnet to the container. This allows containers on the same network to communicate with each other using the assigned IP addresses.

Docker offers the `--network=network` option to connect a container to a specific network. This allows to manage communication between containers located on different networks.

When a container is started in "host" network mode, it directly uses the host network.



Networks

'**docker inspect**' is a key tool for obtaining detailed information about Docker containers, including network details (IP addresses, exposed ports, and other network configurations).

When services are exposed from a Docker container, it is possible to make these services accessible to the host or outside the host through port mapping (port forwarding). The **-p** and **-P** options of the created docker and docker run are used to manage port mapping.

These options are managed by automatically configuring the iptables **NAT** rules on the host.



Networks

By using a **user-defined network** (rather than the bridge network), containers can communicate with each other using their "logical" name, as well as their IP address.

The following command creates a user-defined network:

docker network create -d network-driver network-name

Connecting a container to a network can be as follows:

***docker run --network=my-net --name=container1 -it
busybox***

The other containers connected to this network can see this container by its "logical" name (in our case "container1").

A container can also be connected to multiple networks.



Registry

A **registry** is a service for managing a set of container images. Docker's public registry is **Docker Hub**, while **Docker Registry** is a tool for managing your own private registry and can run as a container.

Here are the main operations of a registry:

- ***docker pull image-name***: download an image from the registry to the host's local cache
- ***docker build***: download the image automatically
- ***docker push image-name***: upload an image to the registry
- ***registry query***



Docker Hub

How to use Docker Hub:

- **login:** `docker login [ -u aswroma3 ] [ -p password ] [ server ]`
- **creation and "tagging" of an image:** `docker build -t aswroma3/myhello .` or `docker build -t myhello .` followed by `docker tag myhello aswroma3/myhello`
- **saving an image on the registry (must be "tagged"):** `docker push aswroma3/myhello`
- **loading an image from the registry (optional):** `docker pull aswroma3/myhello`
- **creation and execution of a container from the image:** `docker run aswroma3/myhello`



Docker Registry

How to use Docker Registry:

- **registry startup** on the **myregistry** node: `docker run -d -p 5000:5000 --restart=always --name registry -v /var/local/docker/registry:/var/lib/registry registry:2`
- **creation and "tagging" of an image**: `docker build -t myhello .` and `docker tag myhello myregistry:5000/myhello`
- **saving an image on the registry** (must be "tagged"): `docker push myregistry:5000/myhello`
- **loading an image from the registry**: `docker pull myregistry:5000/myhello`
- **creation and execution of a container from the image**: `docker run myregistry:5000/myhello`



General recommendations

Recommendations on containers and their images are geared towards maximising efficiency, portability and ease of handling.

- **Only one process per container:**
 - **Reuse images and containers:** maintaining a single process per container allows the reuse of images and containers themselves, making management and scalability easier.
 - **Horizontal scalability:** being monolithic, containers can be replicated horizontally to support load increases without changing the architecture.



General recommendations

- **"Ephemeral" containers:**
 - **Temporariness and stateless:** containers should be designed to be stopped and destroyed easily, so that they can be replaced by new containers quickly.
 - **Support for availability and scalability.**
- **Minimal containers:**
 - **Use small basic images:** choose the lightest basic images possible to reduce container size and improve performance.
 - **Avoid unnecessary packages.**
 - **Minimize the number of layers:** limit the number of layers inside the container image to simplify handling and improve performance.



A containment application

Here is an example of running a simple web application (Spring Boot) in a Docker container: the **lucky-word** application.

```
# Dockerfile for the lucky-word application
FROM openjdk:11-jdk

# Install the application binary
ADD build/libs/lucky-word.jar lucky-word.jar

EXPOSE 8080

# Launch the Java application
ENTRYPOINT ["java", "-Xmx128m", "-Xms128m"]

CMD ["-jar", "lucky-word.jar"]
```



A containment application

How to build and run this application:

- **build the Spring Boot application lucky-word** (in the development environment): *gradle build*
- **build a container image for the application** (in the Docker environment): *docker build --rm -t lucky-word .*
- **create and start the container** (always in the Docker environment): *docker run -p 8080:8080 lucky-word*



A common mistake

Be careful to avoid the following **common mistakes** (which sooner or later all commit).

After editing (the source code of) an application, remember (**always!**) to do the following:

- make (or repeat) the (Java) build of the application
- Build (or repeat) the Docker image (Docker)
- Push (or repeat) the Docker Hub of the Docker image (if necessary)
- sometimes you may also need to remove the previous version of the Docker image from the local cache